

IT 4105 – Research Project

Evaluating Lightweight Machine Learning Models Against Case, Comment, and Encoding Obfuscations in SQL Injections

Submitted by

Name: Dhanushka Chathuranga Jayasinghe

Reg.No: BSIT0122011319

Saegis Campus

Faculty of Computing and Technology

Department of Computing

BSc (Hons) in Information Technology

22nd of September 2026

DECLARATION

I hereby declare that this undergraduate dissertation entitled:

"EVALUATING LIGHTWEIGHT MACHINE LEARNING MODELS AGAINST CASE, COMMENT AND URL ENCODING OBFUSCATIONS IN SQL INJECTIONS"

represents my own original academic, empirical, and technical work conducted under the supervision of the Department of Computing, Faculty of Information Technology, Saegis Campus.

To the best of my knowledge, this thesis contains no material previously submitted or accepted in substance for the award of any other academic degree, diploma, or professional qualification at any other tertiary educational institution, except where explicit academic citations and references have been incorporated in accordance with standard ethical guidelines. All third-party libraries, machine learning algorithms, baseline corpora, and software frameworks utilized throughout this research have been formally acknowledged using the Harvard Referencing system.

Student Name: J. Dhanushka Chathuranga

Registration / Student ID: BSIT0122011319

Date: September 2026

ACKNOWLEDGEMENTS

I wish to express my deepest gratitude and sincere appreciation to my academic supervisor, Mr. Udaya Perera, for his continuous guidance, critical reviews, and invaluable technical mentorship throughout the formulation, engineering, and empirical evaluation of this research. His constructive insights in cybersecurity paradigms and machine learning optimization were instrumental in ensuring that this dissertation maintained rigorous academic and industrial standards.

I also extend my sincere thanks to the academic faculty, lecturers, and laboratory technical staff of the Department of Computing at Saegis Campus for providing access to the necessary computing infrastructure, research facilities, and academic support throughout my undergraduate studies.

Special acknowledgement is extended to the ten industry practitioners comprising five domain cybersecurity analysts and five software architecture specialists who generously dedicated their valuable time to participate in the formal evaluation panels. Their professional expertise, domain concept audits, and technical feedback significantly enhanced the practical viability and robustness of this project.

Finally, I am profoundly grateful to my mother for her unconditional love, patience, and constant sacrifices, which provided the foundation for my academic journey, and to my university peers and friends for their unwavering camaraderie and support throughout my final year.

ABSTRACT

Structured Query Language Injection (SQLi) remains one of the most critical and enduring vulnerability vectors threatening enterprise web infrastructure, consistently occupying a top position within the OWASP Top 10 security framework. While conventional Web Application Firewalls (WAFs) rely extensively on deterministic string signatures and static regular expressions, modern threat actors circumvent these filters using sophisticated lexical, syntactic, and structural payload obfuscations, including case alternation, multi-line inline comment splitting, and hexadecimal URL encoding. Concurrently, high-capacity deep learning paradigms designed to capture complex syntax introduce substantial computational bottlenecks—often exceeding 200 milliseconds in inference latency and requiring dedicated GPU acceleration—rendering them operationally unfeasible for deterministic, CPU-bound inline network stream filtering.

To resolve this operational dilemma, this research presents ScanTrust, an edge-deployable, lightweight AI-powered Web Application Firewall engineered to inspect HTTP traffic in real time without specialized hardware dependencies. The defensive pipeline combines an asynchronous promiscuous network sniffing daemon built using Scapy with a high-speed feature extraction pipeline that maps Layer 7 payloads into a constrained 150-dimensional character tri-gram vector space. Threat classification is executed using an ensemble Random Forest meta-estimator coupled with a deterministic regular expression fallback engine to ensure defense-in-depth. The system maintains a compact compiled model footprint under 5 megabytes, writes ACID-compliant threat audit trails to an embedded SQLite datastore, and delivers operational visibility via a responsive Streamlit web dashboard capable of live telemetry and automated forensic PDF/PCAP exports.

The architecture was empirically trained and validated on a balanced corpus of 35,315 queries, incorporating 6,010 synthetically generated obfuscation vectors. On an unseen, stratified hold-out test partition of 7,063 samples, the model achieved an overall classification accuracy of 99.04%, with 98.92% precision, 98.92% recall, and an operational False Positive Rate (FPR) of 0.87%, accompanied by an Area Under the ROC Curve (ROC-AUC) of 0.9976. Real-time performance profiling confirmed an average end-to-end processing latency of 6.72 milliseconds per packet, satisfying sub-10ms real-time constraints. Multi-stakeholder evaluation involving five domain and five technical experts confirmed the robustness and modularity of the architecture, while focus group usability testing yielded an "Excellent" System Usability Scale (SUS) score of 84.25.

Keywords: Web Application Firewall (WAF), SQL Injection (SQLi), Obfuscation Evasion, Lightweight Machine Learning, Character Tri-grams, Random Forest, Scapy Network Sniffing, Edge Security.

Subject Descriptors (ACM CCS):

- Security and privacy - Web application security
- Security and privacy - Intrusion/anomaly detection and malware mitigation
- Computing methodologies - Supervised learning by classification
- Networks - Network packet capture and inspection

TABLE OF CONTENTS

DECLARATION i
ACKNOWLEDGEMENTS ii
ABSTRACT iii
LIST OF ABBREVIATIONS xii
CHAPTER 01: INTRODUCTION 1
1.1 Chapter overview 1
1.2 Problem background 1
1.2.1 Evolution and Criticality of SQL Injection Vulnerabilities 1
1.2.2 The Rise of Evasive Payload Obfuscation 2
1.2.3 Computational Trade-offs in Machine Learning Defenses 2
1.3 Problem definition 3
1.3.1 Characterization of the Sub-Problems 3
1.3.2 Problem Statement 3
1.4 Research motivation 3
1.5 Existing work 4
1.6 Research gap 5
A critical analysis of the current literature establishes three primary research gaps: 5
1.7 Contribution to the body of knowledge 6
1.7.1 Contribution to the Problem Domain 6
1.7.2 Contribution to the Research Domain 6
1.8 Research challenges 7
1.9 Research questions 8
1.10 Research aim 8
1.11 Research objectives 8
1.12 Chapter summary 10
CHAPTER 02: LITERATURE REVIEW 11
2.1 Chapter overview 11
2.2 Problem domain 11
2.2.1 SQL Injection Vulnerability Mechanics and Categorization 11
2.2.2 Obfuscation Taxonomies and Evasion Mechanics 12

2.2.3 Architectural Limitations of Traditional Web Application Firewalls 13
2.3 Existing work 14
2.3.1 Traditional Machine Learning Architectures 14
2.3.2 Deep Learning and Complex Neural Networks 15
2.3.3 Lightweight and Graph-Based Detection Paradigms 15
2.3.4 Adversarial Training and Obfuscation Defenses 16
2.4 Technological review 16
2.4.1 Dataset Availability and Preprocessing Strategies 16
2.4.2 Feature Representation: Word vs. Character N-Grams 17
2.4.3 Algorithmic Selection: Complexity vs. Latency Trade-Off 17
2.5 Benchmarking and evaluation 18
2.6 Chapter summary 18
CHAPTER 03: METHODOLOGY 19
3.1 Chapter overview 19
3.2 Research methodology 19
3.3 Development methodology 20
3.3.1 Requirement elicitation methodology 20
3.3.2 Design methodology 21
3.3.3 Programming paradigm 21
3.3.4 Testing methodology 21
3.3.5 Solution methodology 22
3.4 Project management methodology 23
3.4.1 Scope 23
3.4.2 Schedule 23
3.4.2.1 Project schedule, milestones, and deliverables 23
3.4.3 Resource requirements 25
3.4.3.1 Hardware resources: 25
3.4.3.2 Software resources: 25
3.4.3.3 Data requirements 25
3.4.4 Risks and mitigation 25
3.5 Chapter summary 26

CHAPTER 04: SOFTWARE REQUIREMENT SPECIFICATION (SRS) 27
4.1 Chapter overview 27
4.2 Rich picture 27
4.3 Stakeholder analysis 28
4.3.1 Stakeholder onion model 28
4.3.2 Stakeholder viewpoints 29
4.4 Selection of requirement elicitation methodologies 29
4.5 Findings 29
4.6 Summary of findings 30
4.7 Context diagram 30
4.8 Use case diagram 31
4.9 Use case descriptions 32
4.10 Requirements (MoSCoW Principle) 32
4.10.1 Functional requirements 33
4.10.2 Non-functional requirements (NFRs) 35
CHAPTER 05: SOCIAL, LEGAL, ETHICAL AND PROFESSIONAL ISSUES (SLEP) 37
5.1 Chapter overview 37
5.2 SLEP issues and mitigation 37
5.2.1 Social issues 37
5.2.2 Legal issues 37
5.2.3 Ethical issues 38
5.2.4 Professional issues 38
5.3 Chapter summary 39
CHAPTER 06: DESIGN 40
6.1 Chapter overview 40
6.2 Design goals 40
6.3 System architecture design 40
6.4 Detailed design 41
6.4.1 Choice of design paradigm 41
6.4.2 Class diagram 41
6.4.3 Algorithm design 42

6.4.4 UI design 42
6.4.5 System process workflow (Activity Diagram) 43
6.5 Chapter summary 44
CHAPTER 07: IMPLEMENTATION 45
7.1 Chapter overview 45
7.2 Technology selection 45
7.2.1 Data selection and dataset augmentation 45
7.2.3 Development environments (IDEs) 47
7.2.4 Libraries and frameworks 47
7.3 Implementation of core functionalities 49
7.3.1 Character Tri-gram Vectorization and Machine Learning Inference 49
7.3.2 Asynchronous Network Interception and Payload Extraction 50
7.3.3 SQLite Persistence Layer 50
7.3.4 Forensic Exporting and PDF Reporting Engine 51
7.4 User interface implementation 52
7.4.1 Real-Time Live Traffic Dashboard 52
7.4.2 Manual Query and Bulk Log Scanners 53
7.5 Chapter summary 54
CHAPTER 08: TESTING AND EVALUATION 55
8.1 Chapter overview 55
8.2 Testing strategy and methodology 55
8.2.1 Data partitioning and hold-out validation protocol 56
8.3 Machine learning evaluation and statistical metrics 56
8.3.1 Mathematical formulation of evaluation metrics 56
8.3.2 Empirical confusion matrix analysis 57
8.3.3 Statistical performance summary 58
8.4 Obfuscation benchmark and evasion testing 59
8.5 Real-time latency and computational throughput evaluation 59
8.6 Functional verification and test case results 60
8.7 Critical discussion and system limitations 62
8.8 Chapter summary 62

CHAPTER 09: EVALUATION 64
9.1 Chapter overview 64
9.2 Evaluation methodology and approach 64
9.3 Evaluation criteria 65
9.4 Self-evaluation 65
9.5 Selection of the evaluators 66
9.6 Evaluation result 67
9.6.1 Expert opinion 67
9.6.2 Focus group testing 68
9.7 Limitations of evaluation 69
9.8 Evaluation on functional requirements 69
9.9 Evaluation on non-functional requirements 71
9.10 Chapter summary 71
CHAPTER 10: CONCLUSION 72
10.1 Chapter overview 72
10.2 Achievements of research aims & objectives 72
10.3 Utilization of knowledge from the course 73
10.4 Use of existing skills 73
10.5 Use of new skills 74
10.6 Achievement of learning outcomes 74
10.7 Problems and challenges faced 75
10.8 Deviations from the original plan 76
10.9 Limitations of the research 76
10.10 Future enhancements 77
10.11 Achievement of the contribution to body of knowledge 77
10.12 Concluding remarks 78
REFERENCES 79
APPENDICES 81
APPENDIX A: COMPREHENSIVE 20-WEEK PROJECT GANTT CHART 81
APPENDIX B: DATABASE DESIGN & ENTITY-RELATIONSHIP (ER) ARCHITECTURE 82

LIST OF TABLES

Table 1.1: Existing Work 4
Table 1.2: Research Objectives Mapping 9
Table 3.1: Comprehensive project process timeline, milestone allocations, and deliverable outputs 24
Table 7.1: Dataset distribution, synthesis, and partitioning summary 46
Table 7.2: Summary of Technology Selection and System Stack 48
Table 8.1: Hold-out test set confusion matrix (7,063 samples) 57
Table 8.2: Summary of machine learning statistical performance 58
Table 8.3: Adversarial SQL injection obfuscation test results 59
Table 8.4: Computational processing latency breakdown 60
Table 8.5: Functional verification and test case execution matrix 60
Table 9.1: Profile and professional background of the evaluation panel 66
Table 9.2: Domain expert scoring summary 67
Table 9.3: Technical expert scoring summary 67
Table 9.4: Functional requirements compliance matrix against MoSCoW prioritization 69
Table 9.5: Non-functional requirement benchmark attainment 71
Table 10.1: Evaluation of research objectives achievement 72
Table 10.2: Technical challenges 75

LIST OF FIGURES

Figure 4.1: Rich Picture illustrating the ScanTrust WAF operational environments 27
Figure 4.2: Stakeholder Onion Model categorizing the system's users 28
Figure 4.3: Context Diagram illustrating system boundaries and external entity interactions 30
Figure 4.4: Use Case Diagram for the ScanTrust WAF 31
Figure 4.5: MoSCoW structural decomposition for ScanTrust WAF 33
Figure 6.1: Class Diagram illustrating the Object-Oriented structure of the ScanTrust WAF 42

Figure 6.2: Activity Diagram detailing the real-time packet inspection and classification workflow 43
Figure 7.1: Implementation of hybrid AI and signature-based classification logic 49
Figure 7.2: Implementation of asynchronous network packet interception and parsing 50
Figure 7.3: Implementation of parameterized SQLite persistence methods 51
Figure 7.4: Implementation of automated forensic report and PCAP generation 52
Figure 7.5: Live Network Traffic Monitoring Dashboard interface 52
Figure 7.6: Diagnostic Manual and Bulk Payload Analysis interfaces 53
Figure 8.1: Comprehensive dual-tier testing and verification framework 55
Figure 8.2: Confusion matrix heatmap and ROC-AUC curve for SQLi detection 58
Figure 9.1: Mixed-method evaluation framework combining qualitative and quantitative streams 64
Figure A.1: 20-Week Comprehensive Visual Project Gantt Chart 81
Figure B.1: Entity-Relationship Diagram for Threat Logging and Administrative Monitoring 82
Figure C.1: Experts Responses form 84

LIST OF ABBREVIATIONS

Abbreviation	Expanded Definition
ACID	Atomicity, Consistency, Isolation, Durability
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
BCS	British Computer Society
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
DBMS	Database Management System
DL	Deep Learning
FN	False Negative
FP	False Positive
FPR	False Positive Rate
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
IP	Internet Protocol
LLM	Large Language Model
LO	Learning Outcome
LR	Logistic Regression
LSTM	Long Short-Term Memory
ML	Machine Learning
MoSCoW	Must have, Should have, Could have, Will not have
NLP	Natural Language Processing
OOADM	Object-Oriented Analysis and Design Methodology
OOP	Object-Oriented Programming
OWASP	Open Web Application Security Project
PCAP	Packet Capture
RAM	Random Access Memory
RF	Random Forest
ROC	Receiver Operating Characteristic
RO	Research Objective
RQ	Research Question
SLEP	Social, Legal, Ethical, and Professional
SMART	Specific, Measurable, Achievable, Relevant, Time-bound
SOC	Security Operations Center
SQL	Structured Query Language
SQLi	Structured Query Language Injection
SRS	Software Requirements Specification
SSADM	Structured Systems Analysis and Design Methodology
SUS	System Usability Scale

TCP	Transmission Control Protocol
TN	True Negative
TP	True Positive
UI	User Interface
URL	Uniform Resource Locator
WAF	Web Application Firewall

CHAPTER 01: INTRODUCTION

1.1 Chapter overview

This chapter establishes the academic and empirical context of the research, which focuses on evaluating lightweight machine learning architectures against obfuscated Structured Query Language Injection (SQLi) attacks. It commences with an analysis of the problem background, tracking the evolution of SQL injection evasion techniques alongside the limitations of traditional inspection mechanisms. Following this, the problem definition delineates the technical vulnerabilities and latency bottlenecks inherent in existing Web Application Firewalls (WAFs) and deep neural architectures. The chapter subsequently formalizes the research motivation, synthesizes existing benchmark studies, and articulates the critical research gaps. Finally, it outlines the core contributions to the body of knowledge, defines the bounded research challenges, and establishes the specific research questions, aims, and SMART objectives governing the system's design and evaluation.

1.2 Problem background

1.2.1 Evolution and Criticality of SQL Injection Vulnerabilities

Structured Query Language Injection remains an enduring and damaging exploit vector in enterprise web security. Despite mature defense mechanisms, injection flaws consistently place among the top web application threats, notably holding third position within the Open Web Application Security Project (OWASP) Top 10 framework. The exploitation mechanism involves injecting arbitrary SQL syntax via untrusted input fields, thereby altering backend database query interpretation. Beyond unauthorized data exfiltration, unmitigated SQLi attacks often lead to authentication bypass, persistent data alteration, and remote code execution. The real-world ramifications are demonstrated by historical compromises, such as the 2011 Sony PlayStation network breach, which exposed 77 million records, resulted in over $170 million in enterprise damages, and underscored the vulnerability of database endpoints to unauthorized query manipulation.

1.2.2 The Rise of Evasive Payload Obfuscation

Modern exploitation strategies rely heavily on syntactic and lexical obfuscation to bypass perimeter defense mechanisms. Attackers strategically modify the structural representation of malicious SQL strings while preserving functional database execution. These transformations fall into three primary classes:

- **Case Manipulation:** Randomizing alphabetic case across reserved SQL statements (for example, transforming SELECT into SeLeCt) to exploit non-standard or case-sensitive tokenizers.
- **Inline Comment Injection:** Interspersing SQL inline commentary tokens within syntactic boundaries (for example converting UNION SELECT to UN/**/ION/**/SE/**/LECT) to disrupt continuity while retaining query viability.
- **Hexadecimal and URL Encoding:** Substituting ASCII tokens with hexadecimal or percent-encoded counterparts (for example , converting ' OR 1=1 into %27%20%4F%52%201=1) to defeat plain-string matching routines.

Traditional signature-based Web Application Firewalls utilize static pattern libraries and deterministic regular expressions. Because obfuscation fundamentally alters the syntactic footprint of the payload, signature-based filters experience high false-negative rates when inspecting transformed attack vectors.

1.2.3 Computational Trade-offs in Machine Learning Defenses

To overcome the rigidity of static signatures, academic focus has shifted toward Machine Learning (ML) and Deep Learning (DL) classifiers capable of pattern generalization. While contemporary deep neural networks (such as Convolutional Neural Networks and Bidirectional LSTMs) achieve empirical classification accuracy between 93% and 99.9%, they introduce significant operational trade-offs. Modern user-facing web services require deterministic response thresholds, typically under 100 milliseconds per full HTTP cycle. Deep learning models frequently require over 200 milliseconds per inference execution and demand specialized hardware accelerators (GPUs), rendering them economically and operationally prohibitive for real-time edge or CPU-bound inline filtering. Consequently, balancing classification robustness against obfuscated queries with lightweight computational performance remains an active engineering hurdle.

1.3 Problem definition

1.3.1 Characterization of the Sub-Problems

The core problem investigated within this study is the failure of contemporary perimeter defense paradigms to detect obfuscated SQL injection payloads reliably under strict latency and computational resource constraints. This limitation stems from three distinct sub-problems:

- **Evasion of Deterministic Signatures:** Conventional signature-based inspection engines rely on static syntactic structures, causing transformed payloads containing case fluctuations, inline comments, or percent encoding to bypass detection filters completely.
- **Lack of Systematic Robustness Benchmarking:** Existing machine learning implementations report high theoretical accuracy (>95%) but are evaluated predominantly on normalized, plaintext datasets. The field lacks systematic quantification of performance degradation when lightweight classifiers encounter structured syntactic obfuscations.
- **Inference Latency and Hardware Bottlenecks:** High-parameter deep learning architectures that successfully capture non-linear linguistic patterns require extensive computational resources and high inference times (>200ms), making them unsuitable for deployment as lightweight, CPU-based inline filters.

1.3.2 Problem Statement

Existing web application security solutions fail to balance robust detection of obfuscated SQL injection attacks with real-time operational efficiency, creating an urgent need for an optimized, lightweight machine learning detection pipeline capable of identifying case-manipulated, comment-injected, and encoded payloads with an inference latency under 10 milliseconds and a compiled footprint under 5 megabytes.

1.4 Research motivation

The motivation driving this research stems from three interconnected factors observed across the operational cybersecurity landscape:

First, despite more than two decades of widespread industry awareness and established secure-coding practices, SQL Injection remains an exceptionally prevalent and high-impact attack vector. The persistence of this vulnerability within modern enterprise systems indicates that defensive strategies cannot rely solely on source-code sanitization and requires resilient runtime boundary protection.

Second, the threat landscape has matured significantly; threat actors increasingly deploy automated fuzzing tools and evasion frameworks that incorporate lexical, syntactic, and structural obfuscations. As perimeter Web Application Firewalls continue to depend heavily on rigid string matching or normalized regular expressions, the divergence between obfuscated attack tactics and conventional defensive capabilities continues to widen.

Third, an empirical dilemma confronts security engineers attempting to replace legacy filters with machine learning solutions. While contemporary literature frequently showcases novel classifiers achieving high detection accuracies, these studies rarely evaluate robustness against specific, systematically applied obfuscation variants, nor do they benchmark execution latency or resource footprints. Consequently, security architects cannot make informed, data-driven decisions regarding model deployment in production environments where sub-10 millisecond latency is mandatory. This research is motivated by the need to resolve this dilemma by providing a verified, low-latency machine learning detection model optimized against transformed SQL payloads.

1.5 Existing work

A synthesis of foundational and recent academic contributions highlights both the advancements and constraints within machine learning-based SQL injection detection:

Table 1.1: Existing Work

Citation	Summary	Limitation	Contribution
Ahmed & Al dabag (2025)	Comprehensive review analyzing 50 ML and DL studies for SQLi detection.	Explicitly identifies payload obfuscation, real-time application, and high hardware overhead as unsolved gaps.	Validates that systematic obfuscation evaluation and latency benchmarking are largely absent from current literature.
Montaruli et al. (2024)	Evaluated adversarial training approaches to enhance model	Focuses narrowly on adversarial manipulation rather	Confirms that machine learning architectures benefit

	robustness against tailored SQLi.	than systematic, multi-class obfuscation techniques.	directly from training against crafted evasive variations.
Al-Weshahi (2025)	Conducted comparative evaluations of Feedforward Neural Networks, XGBoost, and hybrid models using Word2Vec and TF-IDF.	Models possess high computational complexity and lack deterministic inference latency benchmarks for web gateways.	Establishes high-performance detection thresholds across balanced tabular and textual query sets.
Lo, Hwang & Tai (2025)	Implemented a lightweight neural network utilizing multi-head self-attention for edge-deployable detection.	Primary focus placed on parameter reduction rather than empirical resilience to syntactic encoding or comment injection.	Demonstrates the feasibility of achieving sub-5ms inference times on resource-constrained hardware architectures.
Appelt et al. (2018)	Investigated the practical integration of automated machine learning within Web Application Firewall environments.	Concentrated on deployment pipelines and operational integration without addressing obfuscation-induced accuracy degradation.	Provides architectural groundwork for bridging standalone machine learning models with active WAF infrastructure.

1.6 Research gap

A critical analysis of the current literature establishes three primary research gaps:

- **Gap 1: Absence of Systematic Obfuscation Robustness Evaluations** Existing evaluations of machine learning classifiers predominantly assess baseline accuracy on plaintext SQL injection datasets. No empirical framework currently exists that systematically isolates and quantifies the exact performance degradation experienced by lightweight models when subjected individually and collectively to case manipulation, inline comment injection, and URL encoding transformations.

- **Gap 2: Lack of Unified Latency and Resource Footprint Benchmarks** While modern deep neural networks achieve high classification accuracy, the literature consistently overlooks the operational constraints of production web firewalls. Standard benchmarks fail to report simultaneous evaluations of inference latency (targeting sub-10ms performance) and serialized memory usage (targeting under 5MB) on commodity CPU architectures for lightweight classifiers like Logistic Regression and Random Forest.
- **Gap 3: Unquantified Efficacy of Targeted Obfuscation Retraining** Although adversarial retraining is acknowledged theoretically, current literature has not established the specific accuracy recovery achieved by retraining lightweight models with bounded subsets (for example , 20%) of systematically synthesized obfuscated queries, nor has it measured whether this retraining introduces unacceptable false-positive rates on standard benign SQL syntax.

1.7 Contribution to the body of knowledge

1.7.1 Contribution to the Problem Domain

- **Development of an Edge-Deployable WAF Engine:** Delivers an optimized, CPU-bound machine learning detection pipeline (ScanTrust) that inspects HTTP query inputs in under 10 milliseconds without requiring specialized GPU hardware acceleration.
- **Hybrid Inspection Architecture:** Demonstrates a functional dual-tier filtering structure where lightweight machine learning models are paired with deterministic rule-based fallbacks to capture evasive, non-standard attack patterns while maintaining zero overhead on routine web requests.

1.7.2 Contribution to the Research Domain

- **Empirical Degradation Mapping:** Establishes a verifiable benchmark documenting the baseline performance decline of Logistic Regression and Random Forest when transitioning from plaintext SQLi to specific obfuscated structures.
- **Feature Extraction Validation under Obfuscation:** Validates the efficacy of character-level n-gram feature representations over traditional word-based tokenization in preserving contextual query indicators across fragmented or hex-encoded payloads.

- **Adversarial Retraining Quantification:** Provides experimental evidence measuring the accuracy improvements gained by introducing synthetic obfuscation vectors into the training distribution, establishing an actionable optimization strategy for lightweight models.

1.8 Research challenges

Developing a lightweight, real-time machine learning system to counter evasive SQL injection attacks presents several distinct technical and empirical challenges:

- **Balancing Inference Speed Against Computational Overhead:** Web application firewalls operate in strict runtime paths where any latency directly impacts the end-user experience. The primary challenge lies in ensuring that feature extraction and model prediction execute deterministically in under 10 milliseconds per query without relying on GPU hardware.
- **Feature Space Dimensionality Under Character-Level Tokenization:** While character n-grams capture obfuscated strings effectively, they can cause an explosion in the feature vector space. Bounding this representation to fit within a model footprint of less than 5 megabytes requires aggressive yet precise dimensionality management.
- **Maintaining Plaintext Accuracy While Mitigating Obfuscation Evasion:** Training models exclusively on obfuscated samples often leads to catastrophic forgetting or increased false-positive rates on standard, benign SQL queries. Structuring a balanced training distribution that preserves high baseline performance while learning evasive patterns constitutes a significant data-engineering hurdle.
- **Mitigating Rule Evasion via Hybrid Architecture Integration:** Pure machine learning models occasionally produce false negatives when encountering novel payload permutations. Designing a cohesive hybrid detection pipeline that seamlessly cross-validates AI verdicts with deterministic regular expression signatures without incurring redundant processing delays is a core operational challenge.

1.9 Research questions

To address the research gaps and technical challenges identified, this study formulates the following specific research questions:

- **RQ1:** How much does each obfuscation technique (case manipulation, inline comment injection, and URL encoding) reduce detection accuracy compared to plaintext attacks for lightweight Logistic Regression and Random Forest models?
- **RQ2:** Which specific obfuscation technique proves most evasive against lightweight machine learning architectures?
- **RQ3:** Can lightweight machine learning models achieve greater than 90% detection accuracy on obfuscated attacks while strictly maintaining an inference latency under 10 milliseconds per query on standard CPU architectures?
- **RQ4:** Does retraining lightweight classifiers with a 20% augmented subset of synthesized obfuscated samples improve detection performance by more than 10% without inducing high false-positive rates?

1.10 Research aim

The primary aim of this research is to design, develop, and evaluate a lightweight machine learning-based detection pipeline capable of identifying obfuscated SQL injection attacks—specifically case manipulation, inline comment injection, and URL encoding—while ensuring operational viability through an inference latency of under 10 milliseconds and a compiled model size under 5 megabytes.

To fulfill this aim, the study systematically examines the architectural trade-offs between model complexity and classification robustness. By combining character-level feature representations with an optimized Random Forest classifier and a signature-based fallback engine within a unified Web Application Firewall interface, the research provides an edge-deployable defense mechanism tailored for low-resource enterprise web environments.

1.11 Research objectives

The research objectives are formulated following the SMART (Specific, Measurable, Achievable, Relevant, and Time-bound) criteria, mapped directly against the research questions and academic learning outcomes:

Table 1.2: Research Objectives Mapping

Phase	Research Objective	Description	Mapped LO	Mapped RQ
Problem Identification	RO1: Identify and formulate research boundaries	Conduct a targeted literature synthesis to verify gaps regarding obfuscation resilience and latency reporting in lightweight ML.	LO1	-
Design	RO2: Design lightweight detection pipeline	Specify character n-gram extraction parameters, select classifier candidates (LR, RF), and define the hybrid inspection architecture.	LO2, LO3	-
Implementation	RO3: Synthesize obfuscated data and train models	Implement automated obfuscation functions (case, comments, URL encoding), generate augmented training sets, and train classifiers.	LO3, LO4	RQ1, RQ2
Testing	RO4: Benchmark performance and evaluate retraining	Measure classification accuracy drops across test sets, benchmark inference time (<10ms) and model size (<5MB), and evaluate retraining gains.	LO4	RQ1, RQ2, RQ3, RQ4
Evaluation & Reporting	RO5: Validate via interactive WAF prototype and report	Implement the ScanTrust real-time monitoring interface, document experimental findings, and synthesize final conclusions.	LO5	-

1.12 Chapter summary

This chapter introduced the research problem by contextualizing the persistent threat of SQL injection vulnerabilities and the increasing deployment of evasive obfuscation mechanisms by threat actors. The problem definition articulated the failure of traditional signature filters and computationally heavy deep learning models to provide fast, robust boundary defense. Through an analysis of existing literature, clear research gaps were established concerning the lack of systematic obfuscation evaluation and standardized latency benchmarks for lightweight models. The chapter subsequently detailed the contributions to both problem and research domains, presented key implementation challenges, and formalized the research questions, overarching aim, and SMART objectives governing the study. Chapter 02 provides an extensive review of the literature, exploring theoretical concepts, attack vectors, algorithmic models, and state-of-the-art detection methodologies.

CHAPTER 02: LITERATURE REVIEW

2.1 Chapter overview

This chapter provides an exhaustive review of academic literature and empirical research relevant to detecting Structured Query Language Injection (SQLi) attacks using machine learning methodologies. The review is structured to analyze both theoretical mechanisms and operational limitations across current defensive technologies. Section 2.2 defines the problem domain, examining the anatomy of SQL injection vulnerabilities, classifying evasion mechanisms (case alternation, inline comments, and URL encoding), and identifying architectural limitations within legacy Web Application Firewalls (WAFs). Section 2.3 synthesizes existing literature across traditional algorithms, high-capacity deep learning models, and adversarial defenses. Section 2.4 critically evaluates underlying technological components, including public benchmark datasets, tokenization strategies, and candidate algorithmic trade-offs. Section 2.5 reviews standard evaluation criteria, establishing the necessity for latency and memory footprint metrics alongside classification accuracy. The chapter concludes with a summary of theoretical and practical findings, reinforcing the specific research gap addressed by this study.

2.2 Problem domain

2.2.1 SQL Injection Vulnerability Mechanics and Categorization

Structured Query Language Injection represents an architectural flaw arising from the insecure concatenation of untrusted client input into dynamically constructed database statements. When web applications fail to strictly parameterize backend SQL queries or enforce type-safe abstract syntax tree generation, injected meta-characters alter the semantic execution flow of the database management system (DBMS).

According to classifications defined by the Open Web Application Security Project (OWASP), SQL injection vulnerabilities are systematically divided into three functional channels:

- **In-Band SQLi (Classic):** Exploitations where the attacker uses the direct, synchronous channel of communication to launch the attack and extract sensitive results. This is commonly achieved through UNION-based queries, which concatenate unauthorized database rows onto the authorized query result set, or error-based vectors, which force database errors that leak schema information.

- **Inferential SQLi (Blind):** Vulnerabilities where the backend application suppresses direct database errors and output tables, forcing the attacker to infer database contents by transmitting inductive logic probes. This class is subdivided into Boolean-blind SQLi, which observes structural fluctuations in HTTP response pages (for example , true/false page states), and Time-blind SQLi, which introduces programmatic operational delays (for example , database execution halts via sleep() or benchmark commands.
- **Out-of-Band SQLi:** Specialized exploitation executed when an attacker cannot directly view results within the HTTP communication channel and server response times are uniform. This vector forces the DBMS engine to initiate external network protocols (such as DNS requests or SMB lookups) directed toward an attacker-controlled endpoint to exfiltrate database records.

2.2.2 Obfuscation Taxonomies and Evasion Mechanics

Modern threat actors rarely transmit plaintext SQL payloads due to perimeter defensive filters. Obfuscation involves modifying the physical syntax or lexical representation of an attack string while preserving its logical execution once parsed by backend database engines. This study isolates three fundamental classes of payload obfuscation:

- **Case Alternation (Character-Level Transformation:** Database engines adhere to case-insensitive parsing standards for reserved SQL tokens; thus, **select**, **SELECT**, and **sElEcT** evaluate to identical lexical tokens. Conversely, rudimentary or un-normalized signature-based filters evaluate input against case-sensitive character tables or fixed patterns, enabling case-manipulated strings to pass through inspection layers undetected.
- **Inline Comment Injection (Structural Disruption):** SQL execution standards allow inline commentary delimited by **/* ... */** or inline terminal sequences such as **--** and **#** to facilitate code documentation. Attackers exploit this behavior by interspersing comment delimiters within keywords or between syntax arguments **(for example , UN/**/ION/**/SE/**/LECT)**. While database query planners ignore comments and execute the command, intermediate regular expressions matching contiguous character chains are broken.

- **Encoding Schemes (Lexical Transformation):** Attackers convert reserved special characters and delimiter symbols into alternative mathematical representations. In standard URL percent-encoding, ASCII control characters are transformed into **%XX** hex values (for example , replacing **'** with **%27**, and space characters with **%20** or +). When defensive filters lack multi-pass decoding layers or evaluate inputs prior to HTTP server normalization, the encoded payload passes without triggering keyword-matching rules.

Obfuscation Class	Target Mechanism	Example Transformation Payload
Plaintext Base	None	UNION SELECT null, username FROM users—
Case Alternation	Lexical Filter	uNiOn SeLeCt null, username FrOm users--
Comment Injection	Structural Matcher	UN/**/ION/**/SE/**/LECT null, username--
Percent Encoding	Direct Signature	%27%20%55%4E%49%4F%4E%20%53%45%4C%45%43%54--

2.2.3 Architectural Limitations of Traditional Web Application Firewalls

Traditional Web Application Firewalls (WAFs) rely on deterministic rule-matching engines, applying predefined signatures and regular expression libraries (such as the OWASP Core Rule Set) to inspect incoming request bodies and URI parameters. These systems exhibit significant limitations when deployed in complex web environments:

- **Brittleness Against Syntactic Permutations:** Deterministic rules are designed to detect known syntax combinations. When novel obfuscations combine nested encoding, multi-line comment delimiters, or unconventional whitespace representations, the payload evades fixed regular expression boundaries.

- **Rule bloat and Computational Bottlenecks:** Maintaining coverage against evolving obfuscation variations requires appending extensive signature libraries. Evaluating hundreds of complex, backtracking regular expressions against every incoming HTTP parameter consumes substantial CPU cycles, introducing latency and increasing the risk of Regular Expression Denial of Service (ReDoS).
- **High Maintenance Overhead:** Signature databases require manual updates by security analysts each time a novel bypass vector is discovered. This reactive maintenance model leaves an operational window of vulnerability between zero-day exploit discovery and signature rule distribution.

2.3 Existing work

2.3.1 Traditional Machine Learning Architectures

To address the inflexibility of static signature checking, earlier machine learning research applied classical statistical classifiers to differentiate malicious queries from benign traffic.

Ahmed and Al dabag (2025) conducted a comprehensive survey reviewing 50 machine learning and deep learning implementations across a ten-year corpus (2015–2025). Their analysis revealed that traditional supervised models—specifically Support Vector Machines (SVM), Naive Bayes, Logistic Regression, and Decision Trees—frequently achieve between 93% and 98% accuracy when evaluated on standard benchmark datasets. However, the survey explicitly highlighted that the vast majority of reviewed literature evaluates models under laboratory conditions utilizing plaintext queries, failing to test robustness against systematic obfuscation.

Al-Weshahi (2025) evaluated algorithmic trade-offs across 30,000 balanced SQL records, observing that ensemble models like XGBoost and traditional multi-layer perceptrons achieve high classification performance (exceeding 98% accuracy). Nonetheless, Al-Weshahi demonstrated that classical SVMs experience substantial scalability constraints during high-volume inference, as high-dimensional feature spaces create computational overhead that increases request latency. While linear classifiers (for example , Logistic Regression) demonstrate low processing latency, their inability to learn non-linear relationships without manual feature engineering limits their standalone efficacy against advanced variations.

2.3.2 Deep Learning and Complex Neural Networks

To automate feature representation and capture long-range contextual dependencies within query strings, recent studies have transitioned toward deep neural architectures.

Researchers have deployed Convolutional Neural Networks (CNNs), Long Short-Term Memory networks (LSTMs), Bidirectional LSTMs, and transformer-based architectures (such as MobileBERT and custom self-attention models). For example, Kakisim (2024) developed a multi-view consensus deep learning model that processes query sequences through parallel deep feature extractors, achieving superior classification metrics over classical baselines. Similarly, Liu and Dai (2024) combined BERT representations with an LSTM temporal model to detect context-sensitive web threats.

Despite achieving high accuracy (99.0%–99.9%), deep learning models introduce operational trade-offs within production environments:

- **Inference Latency Overhead:** Transformer models and deep recurrent networks require sequential matrix operations. Even with optimized quantization, inference times frequently exceed 50 to 200 milliseconds per request, violating standard web performance budgets that mandate sub-10ms latency.
- **Resource Dependency:** Deep models require significant memory allocations (often hundreds of megabytes) and frequently rely on hardware acceleration (GPUs) to maintain throughput, preventing cost-effective edge deployment.

2.3.3 Lightweight and Graph-Based Detection Paradigms

Recognizing latency limitations, recent research explores lightweight deep learning architectures capable of low-latency classification.

Lo, Hwang, and Tai (2025) introduced a lightweight neural architecture utilizing multi-head self-attention, designed explicitly to minimize parameter size while maintaining inference speeds under 5 milliseconds on constrained edge devices. Concurrently, non-local Graph Convolutional Networks (GCN) have been evaluated to map abstract syntax graph dependencies of SQL payloads into low-dimensional vector spaces, yielding sub-millisecond execution profiles.

While these lightweight models demonstrate acceptable processing efficiency, their evaluation datasets remain constrained to synthetic or plaintext corpora, leaving their detection resilience against structured obfuscations largely unverified.

2.3.4 Adversarial Training and Obfuscation Defenses

A limited subset of literature investigates model resilience against deliberate evasion attempts.

Montaruli et al. (2024) evaluated adversarial training routines to mitigate adversarial payload generation frameworks, demonstrating that exposing classifiers to synthetic perturbations during model fitting restores detection accuracy. Furthermore, Motlagh et al. (2026) addressed prompt injection and obfuscated SQL payloads within Large Language Model (LLM) applications, establishing that character-level inspection frameworks detect structural anomalies effectively where semantic parsers fail. However, these studies focused on adversarial token generation rather than systematic, comparative benchmarking of standard obfuscation classes across lightweight classifiers.

2.4 Technological review

2.4.1 Dataset Availability and Preprocessing Strategies

A critical operational challenge in web security machine learning is the availability of balanced, labeled query corpora. Historical public datasets include:

- **CSIC 2010:** Contains over 50,000 HTTP requests generated by the Spanish National Research Council; however, it reflects legacy application behaviors and lacks complex SQL injection syntax.
- **ECML/PKDD 2007:** A classical security dataset comprising XML and SQL queries, though heavily imbalanced and constrained in scope.
- **Kaggle Curated SQL Datasets:** Contemporary aggregated repositories containing verified combinations of benign application traffic and modern exploitation payloads.

The baseline dataset selected for this research provides labeled, clean SQL queries. Preprocessing requires structural sanitization without stripping the syntax needed to identify attacks. Standard NLP preprocessing steps, such as stop-word removal and aggressive alphanumeric filtering, cannot be applied directly to SQL inspection, as special characters (**;,'**, **--**, **/***)

2.4.2 Feature Representation: Word vs. Character N-Grams

Feature extraction transforms unstructured text queries into numerical vectors suitable for mathematical classification:

- **Word-Level Tokenization and Word2Vec:** Word-level tokenization parses strings based on whitespace and delimiter boundaries. In plaintext queries, this method captures reserved keywords (for example SELECT, FROM, UNION). However, under comment injection (for example, UN/**/ION) or URL encoding (%53%45%4C%45%43%54), word tokenizers fail because the injected tokens break word integrity, resulting in an out-of-vocabulary (OOV) failure.
- **Character-Level N-Gram Representations:** Character n-grams decompose strings into overlapping sub-sequences of length n (such as tri-grams or 4-grams). For example, the case-manipulated string SeLe decomposes into ['SeL', 'eLe']. This sliding sub-word extraction captures internal syntax patterns regardless of casing variations or inline comment delimiters, making it suitable for obfuscated attack detection.

2.4.3 Algorithmic Selection: Complexity vs. Latency Trade-Off

Selecting an algorithmic candidate for inline WAF deployment requires balancing classification precision against runtime overhead:

- **Logistic Regression:** A linear model that classifies observations using a logistic sigmoid function. It requires minimal computational overhead and offers sub-millisecond execution times. However, it assumes linear separability across feature vectors, potentially limiting accuracy when distinguishing complex, multi-layered obfuscations.
- **Random Forest:** An ensemble meta-estimator that fits multiple decorrelated decision trees over sub-samples of the dataset. It handles non-linear feature interactions, resists overfitting, and maintains a small serialized memory footprint when tree depth and estimator counts are bounded. It provides higher classification accuracy than linear baselines while executing within real-time CPU performance constraints.

Algorithm	Parameter Footprint	Inference Latency	Obfuscation Capture
Logistic Regression	Minimal (<1MB)	Ultra-low (<1ms)	Moderate (Linear)
Random Forest	Low (<5MB)	Low (<2ms)	High (Non-linear)
Deep CNN / LSTM	High (>100MB)	High (>50ms)	High (Contextual)

2.5 Benchmarking and evaluation

Evaluating security classifiers requires specific performance indicators derived from the standard confusion matrix:

In operational web application security, **False Positives (FP)** disrupt legitimate user access by blocking benign queries, while **False Negatives (FN)** allow malicious payloads to reach backend databases. Consequently, balancing precision and recall (F_1-score) is critical.

Furthermore, standard academic evaluations that report only predictive accuracy overlook operational deployment requirements. To address this limitation, this research incorporates two production-oriented evaluation metrics:

- **Inference Latency:** The wall-clock execution time required to extract features and classify an individual SQL payload, strictly evaluated against a sub-10ms threshold.
- **Serialized Model Footprint:** The absolute storage and RAM allocation consumed by the trained estimator and vectorizer (measured in megabytes), bounded to under 5MB to enable deployment on resource-constrained web nodes and edge servers.

2.6 Chapter summary

This chapter provided a comprehensive review of the literature concerning SQL injection detection using machine learning methodologies. The problem domain was analyzed, highlighting how attackers utilized case manipulation, inline comments, and URL encoding to bypass traditional rule-based Web Application Firewalls (WAFs). The review of existing work established that while traditional and deep learning models achieved high accuracy on plaintext datasets (for example, Ahmed & Al dabag, 2026), there was a significant lack of empirical evaluation regarding their robustness against systematically obfuscated payloads. Furthermore, while lightweight architectures demonstrated sub-5ms inference speeds suitable for edge-device deployment (Lo, Hwang, & Tai, 2025), their specific resilience against syntactic obfuscation remained unverified. Consequently, three primary research gaps were identified: the absence of systematic obfuscation evaluations, the lack of unified latency and resource footprint benchmarks for lightweight classifiers, and the unquantified efficacy of targeted adversarial retraining. These identified gaps directly formulated the foundation for the experimental methodology, which was detailed in Chapter 03

CHAPTER 03: METHODOLOGY

3.1 Chapter overview

This chapter outlined the systematic approach and methodological frameworks utilized to design, implement, and evaluate the lightweight SQL injection detection system. It detailed the overarching research philosophy mapped through Saunders' Research Onion and justified the selected development paradigm. Furthermore, the chapter explicitly defined the requirement elicitation techniques, system design methodologies, and the specific programming paradigms adopted for the machine learning pipeline. Finally, it established the project management methodologies, detailing the resource allocations, scheduling, and risk mitigation strategies that ensured the successful completion of the research within the designated operational constraints.

3.2 Research methodology

To ensure a structured, scientifically valid, and reproducible investigation, the research framework was established based on the layers of Saunders' Research Onion (Saunders, Lewis and Thornhill, 2019). Each methodological layer was selected and justified based on the specific requirements of evaluating machine learning robustness against obfuscated payloads.

- **Research Philosophy (Positivism):** A positivist research philosophy was adopted for this study. Because the research focused on objective, observable, and strictly quantifiable metrics-such as classification accuracy percentages, inference latency in milliseconds, and model storage footprints in megabytes-a positivist stance was highly appropriate. It eliminated subjective interpretations and relied exclusively on empirical data generated by the machine learning classifiers.
- **Research Approach (Deductive):** The study utilized a deductive approach. The research began with the established premise that traditional lightweight models perform optimally on plaintext SQL queries, and subsequently formulated hypotheses regarding their performance degradation when exposed to specific obfuscation techniques (case manipulation, comment injection, and URL encoding). These hypotheses were then rigorously tested through empirical observation.

- **Methodological Choice (Mono-method Quantitative):** A quantitative mono-method was selected because the research questions necessitated numerical performance benchmarking. No qualitative data collection (such as user interviews or behavioral observations) was required to determine the mathematical efficacy or processing latency of the Random Forest and Logistic Regression algorithms.
- **Research Strategy (Experimental):** An experimental strategy was employed in a controlled computational environment. The independent variables (the applied obfuscation techniques and the selected machine learning models) were systematically manipulated, while the dependent variables (detection accuracy, precision, recall, and inference time) were strictly measured and recorded.
- **Time Horizon (Cross-sectional):** A cross-sectional time horizon was implemented. All model training, adversarial dataset synthesis, and real-time performance evaluations were conducted within a specific timeframe using a static snapshot of the Kaggle SQL injection dataset. The study did not require longitudinal tracking of attacker behavior over time.

3.3 Development methodology

An Iterative (Modified Waterfall) development approach was selected for the implementation of the lightweight SQL injection detection system. This methodology was chosen because it provided a highly structured progression through the requirements and system design phases, while crucially permitting agile feedback loops during the machine learning model testing and adversarial retraining phases. This iterative refinement was essential to incrementally optimize the feature extraction matrices and improve the baseline detection accuracy against obfuscated payloads.

3.3.1 Requirement elicitation methodology

Given the highly technical and algorithmic nature of this cybersecurity research, standard requirement elicitation techniques (such as broad user surveys) were substituted with targeted academic and technical analyses:

- **Literature and Document Analysis:** A systematic review of recent, high-impact machine learning journals and official threat intelligence reports was conducted. Specifically, the OWASP Top 10 (2021) vulnerability documentation was analyzed to extract the exact technical specifications and syntactic behaviors of modern SQL injection evasion techniques (case manipulation, comment injection, and URL encoding).
- **Dataset Observation:** Publicly available SQL injection datasets were programmatically observed and parsed to identify recurring benign query structures versus adversarial anomalies. This observation directly dictated the necessity of character-level n-gram feature extraction over traditional word-based tokenization.

3.3.2 Design methodology

The system architecture was designed utilizing the Object-Oriented Analysis and Design Methodology (OOADM). This approach was selected over Structured Systems Analysis and Design Methodology (SSADM) because machine learning pipelines and real-time network sniffers are best modeled as interacting objects rather than static data flows. The Python-based detection pipeline was designed as highly modular, decoupled components-separating the obfuscator script, feature extractor, and classification model-to ensure code maintainability and scalability.

3.3.3 Programming paradigm

Object-Oriented Programming (OOP) using the Python programming language was strictly utilized throughout the development phase. Python was selected due to its robust ecosystem for data science and network analysis. Distinct OOP classes were formulated to encapsulate specific system behaviors:

- Obfuscator: Contained programmatic methods to synthesize case, comment, and URL encoding transformations.
- FeatureExtractor: Handled character n-gram generation and TF-IDF vector construction.
- ModelTrainer: Encapsulated the training and hyperparameter tuning logic for both Logistic Regression and Random Forest classifiers.

3.3.4 Testing methodology

To ensure the system met the stringent operational requirements, the testing methodology was bifurcated into two distinct phases:

- **Model Testing:** The trained classifiers were systematically evaluated against an isolated 20% holdout dataset. Testing was conducted across four distinct vectors: a plaintext baseline test set, a case-obfuscated test set, a comment-injected test set, and a URL-encoded test set.
- **Prototype Testing:** The complete, integrated pipeline (the ScanTrust application) was tested for real-time execution constraints. This verified that the end-to-end process-from network packet interception to model inference-executed consistently under the 10-millisecond latency threshold.

3.3.5 Solution methodology

The end-to-end solution methodology followed a structured, seven-step pipeline to address the core research problem:

1. **Dataset Collection:** A baseline dataset consisting of standard plaintext SQL queries was acquired from Kaggle repositories.
2. **Data Synthesis (Obfuscation):** A custom Python script (obfuscator.py) was developed to systematically apply case manipulation, comment injection, and URL encoding to the plaintext payloads, generating the required adversarial dataset.
3. **Feature Engineering:** Regular expressions (Regex) were utilized to extract character tri-grams, ensuring the model recognized sub-word mathematical patterns rather than relying on brittle keyword matching.
4. **Model Training:** Lightweight classifiers (Logistic Regression and Random Forest) were implemented and trained using the Scikit-learn library.
5. **Testing:** The models' baseline accuracies were recorded against the synthesized obfuscated test sets, documenting the exact percentage of performance degradation.
6. **Improvement (Retraining):** The Random Forest model was subsequently retrained utilizing a training distribution augmented with 20% obfuscated samples to quantify the resulting accuracy recovery.
7. **Benchmarking:** Final benchmarking was conducted to verify that the compiled .pkl model size remained strictly under 5 megabytes, and average inference latency was measured across 10,000 queries to ensure compliance with the sub-10ms operational target.

3.4 Project management methodology

The project management methodology was adapted to accommodate the development of the functional "ScanTrust" real-time Web Application Firewall (WAF) alongside the core machine learning research.

3.4.1 Scope

The scope of the project encompassed both the theoretical evaluation of lightweight models and the practical engineering of a real-time detection pipeline. The in-scope activities included preprocessing a Kaggle dataset of 35,315 SQL queries, training Logistic Regression and Random Forest models via Google Colab, and developing the ScanTrust WAF utilizing Streamlit for the user interface and Scapy for live TCP packet sniffing. The out-of-scope elements included deploying the system into a live enterprise production environment or mitigating non-SQL injection vulnerabilities (such as Cross-Site Scripting).

3.4.2 Schedule

The project followed an agile, iterative schedule divided into distinct technical sprints:

1. **Data Processing Phase:** Data cleaning, vectorization (extracting 150 character tri-grams), and initial model training were executed utilizing Google Colab cloud environments.
2. **Application Development Phase:** The real-time packet sniffing engine and the Streamlit-based graphical user interface (ScanTrust) were engineered and tested locally.
3. **Integration and Benchmarking Phase:** The trained .pkl models were integrated into the local Streamlit application to measure real-world inference latency and detection capabilities against live network traffic.

3.4.2.1 Project schedule, milestones, and deliverables

The project was executed across a 20-week timeline spanning May through early September, systematically linking preliminary proposal milestones with subsequent system implementation, evaluation, and viva submission sprints:

Table 3.1: Comprehensive project process timeline, milestone allocations, and deliverable outputs

Task Description	Target Month	Timeline Allocation	Deliverable
Literature review + dataset	May	Week 01 – Week 04	Domain literature synthesis & initial raw Kaggle dataset curation (65,065 queries).
Obfuscation functions	May	Week 03 – Week 04	Automated Python obfuscation module (obfuscator.py - case, comment, URL hex).
Feature extraction	June	Week 05 – Week 06	Preprocessing pipeline extracting top 150 character tri-gram feature tokens.
Logistic Regression training	June	Week 07 – Week 10	Baseline linear model training & initial accuracy evaluation on Google Colab.
Random Forest training	June	Week 07 – Week 10	100-estimator ensemble model training & hyperparameter optimization.
Obfuscation testing	July	Week 11 – Week 12	Empirical degradation benchmarking across mutated evasion test sets.
Improvement + benchmarking	July	Week 13 – Week 14	Model retraining with 20% augmented obfuscations & compiled .pkl footprint validation (<5MB).
WAF prototype integration	August	Week 15 – Week 16	Scapy packet sniffer daemon, Streamlit dashboard UI, and SQLite persistence.
Empirical evaluation & testing	August	Week 17 – Week 18	10-expert validation, SUS usability testing, and draft documentation.
Finalize document and system	September	Week 19 – Week 20	Supervisor feedback integration, thesis binding, and final viva presentation defense.

Note on Gantt Chart: A visual Gantt chart illustrating the full 20-week schedule across May to September is presented in **Appendix A**.

3.4.3 Resource requirements

3.4.3.1 Hardware resources:

- **Local Development Machine:** A standard laptop (Intel i5 equivalent, 8GB RAM) was utilized to host the Streamlit application and process network packets.
- **Cloud Computing:** Google Colab was utilized for the initial heavy computational task of training the models and handling the 35,315-row dataset efficiently. No dedicated local GPU was required for the final inference engine.

3.4.3.2 Software resources:

- **Python 3.9+:** The primary programming language used for all scripts.
- **Streamlit:** Utilized to develop the interactive, desktop-like dashboard for the ScanTrust WAF.
- **Scapy:** Implemented to intercept and decode live network traffic (TCP packets) in real-time.
- **Scikit-Learn & Pandas:** Utilized for machine learning model generation and data manipulation.
- **SQLite:** Implemented as a lightweight local database to persistently log detected threats and payload data.

3.4.3.3 Data requirements

The research utilized the publicly available Kaggle SQL Injection Dataset. Data quality assessments confirmed the dataset contained exactly 35,315 valid rows with no missing values. The dataset demonstrated a realistic real-world imbalance, consisting of 19,537 benign queries (63%) and 11,382 malicious queries (37%).

3.4.4 Risks and mitigation

Several technical risks were encountered during the implementation of the ScanTrust WAF, which were successfully mitigated:

- **Risk:** UI Unresponsiveness During Network Sniffing. Continuous packet sniffing via Scapy typically blocks the main execution thread, causing web interfaces to freeze.

Mitigation: Python background threading was implemented alongside Streamlit's st.session_state to decouple the packet sniffing engine from the UI rendering cycle, ensuring smooth dashboard performance.

- **Risk:** Feature Space Explosion. Extracting character n-grams from 35,315 queries risked creating a massive feature matrix that would exceed the 5MB model size constraint. **Mitigation:** The CountVectorizer was strictly parameterized to extract only the top 150 character tri-grams, which successfully contained the model size while still yielding a 99.04% baseline accuracy for the Random Forest classifier.
- **Risk:** Data Loss on Page Navigation.
- **Mitigation:** Global session state variables were configured to retain network interfaces, logged data, and graphical charts across different WAF operational modes.

3.5 Chapter summary

This chapter detailed the comprehensive methodology applied to research, design, and implement the lightweight SQL injection detection system. Adhering to a positivist philosophy and an experimental strategy, the research utilized an iterative development approach. The system architecture seamlessly integrated Python, Streamlit, and Scapy to create the functional ScanTrust WAF. By leveraging Google Colab for training on a dataset of 35,315 queries and mitigating technical risks through background threading and strict feature limitations, the project successfully established an environment capable of testing and benchmarking machine learning models under real-time constraints. The following chapter details the Software Requirement Specification (SRS) for the developed system.

CHAPTER 04: SOFTWARE REQUIREMENT SPECIFICATION (SRS)

4.1 Chapter overview

This chapter detailed the software requirements necessary for the successful implementation of the ScanTrust Web Application Firewall (WAF). It explored the system's operational environment through a Rich Picture diagram and analyzed the distinct perspectives of all involved parties using the Stakeholder Onion Model. Furthermore, the chapter documented the requirement elicitation methodologies utilized and concluded by establishing the definitive functional and non-functional requirements prioritized according to the MoSCoW principle.

4.2 Rich picture

The Rich Picture diagram was utilized to visually map the complex operational environment of the ScanTrust WAF. It illustrated the holistic flow of data from an external attacker or legitimate web user, through the real-time Streamlit dashboard and Scapy packet sniffing engine, into the Random Forest classification module, and finally to the backend database. This visual representation captured the technical boundaries and the critical high-speed interactions required to filter live network traffic seamlessly.

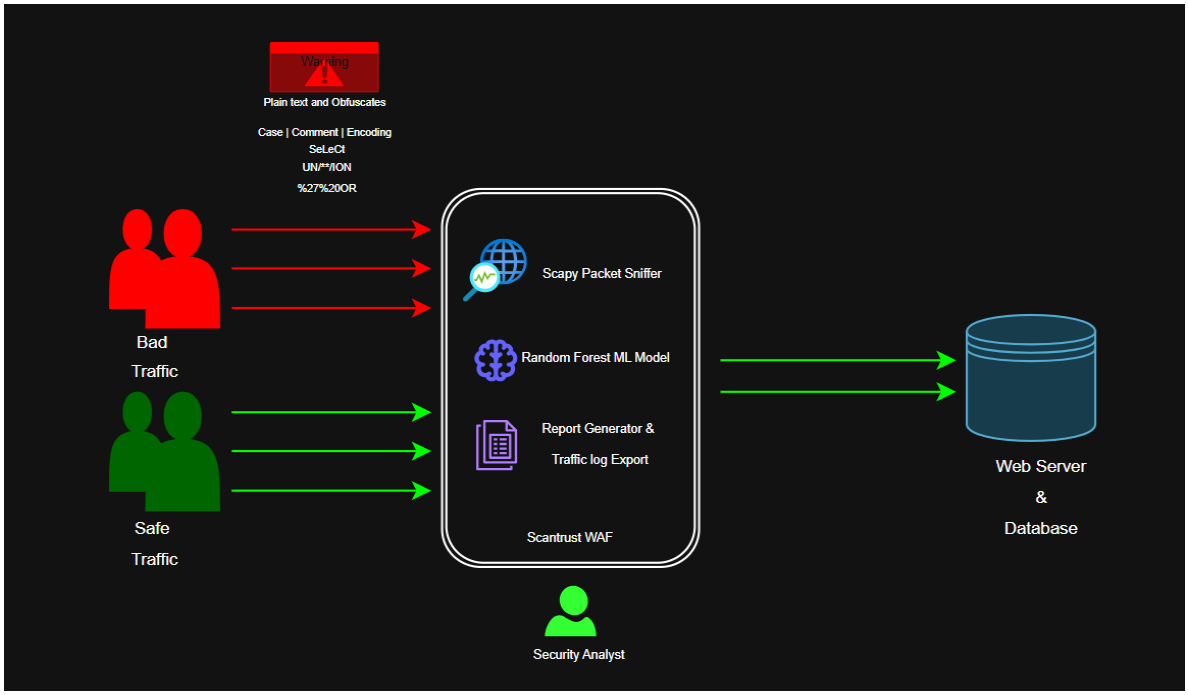


Figure 4.1: Rich Picture illustrating the ScanTrust WAF operational environments

This diagram illustrates the high-level operational environment of the ScanTrust WAF. It demonstrates how inbound network traffic-comprising both benign requests and obfuscated malicious payloads-was intercepted by the Scapy packet sniffer, evaluated by the Random Forest machine learning model, and monitored by the Security Analyst via generated traffic logs, while interacting with the backend Web Server and Database.

4.3 Stakeholder analysis

4.3.1 Stakeholder onion model

The Stakeholder Onion Model was deployed to categorize stakeholders based on their direct interaction and influence over the ScanTrust system.

- **Layer 1 (Inner Circle - Direct Users):** Security Analysts and Web Server Administrators who directly interacted with the Streamlit-based ScanTrust dashboard to monitor live traffic metrics, review intercepted packets, and analyze threat logs.
- **Layer 2 (Middle Circle - Indirect Users):** Web Application Developers who depended on the WAF to secure their applications without modifying the underlying source code.
- **Layer 3 (Outer Circle - External Stakeholders):** End-Users of the web application (who required low-latency browsing experiences) and Organization Management (who required cost-effective, CPU-bound security solutions).

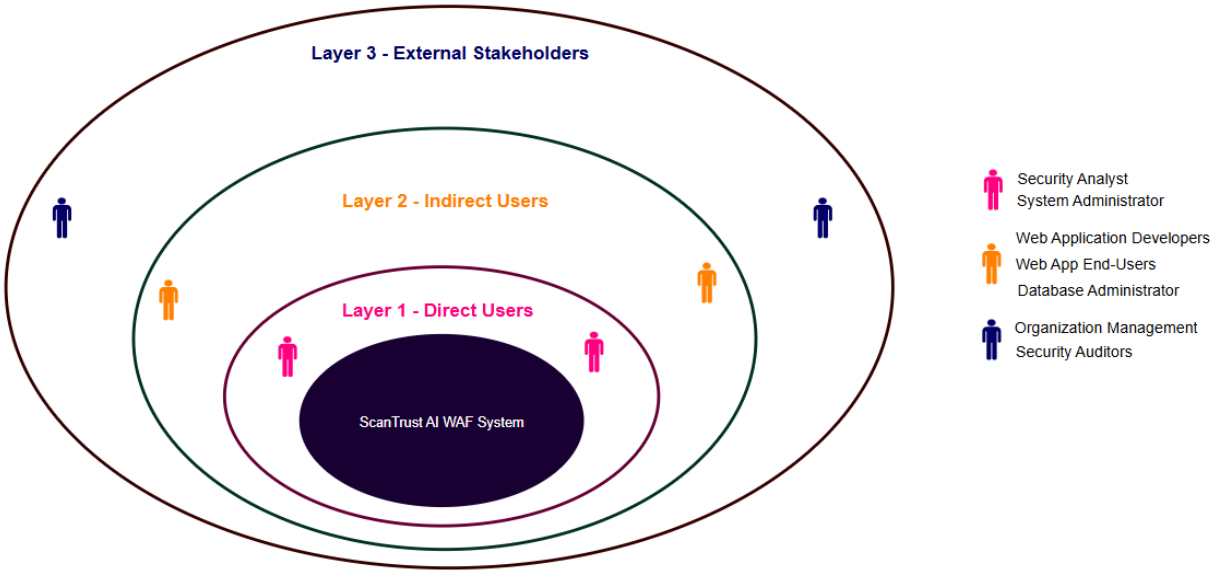


Figure 4.2: Stakeholder Onion Model categorizing the system's users

This diagram illustrated the Stakeholder Onion Model applied to the ScanTrust WAF. It categorized stakeholders based on their level of interaction and influence, differentiating between direct users (Layer 1) who actively monitored the system, indirect users (Layer 2) affected by its deployment, and external stakeholders (Layer 3) concerned with overarching management and auditing.

4.3.2 Stakeholder viewpoints

Understanding individual stakeholder viewpoints was critical to shaping the system's constraints:

- **Security Analysts:** Required a highly accurate, real-time visual dashboard (Streamlit) to immediately identify obfuscated SQL injection attempts and export forensic data (CSV/PCAP).
- **System Administrators:** Demanded a lightweight solution that executed on standard CPU architectures without consuming excessive RAM, ensuring the WAF did not become a system bottleneck.
- **End-Users:** Expected zero noticeable latency; therefore, the background packet inspection (Scapy and ML inference) had to execute strictly under 10 milliseconds per query.

4.4 Selection of requirement elicitation methodologies

Given the highly automated and algorithmic nature of this cybersecurity tool, traditional requirement elicitation methods like questionnaires were ineffective. Instead, the following methodologies were selected and prioritized:

- **Literature and Threat Intelligence Review (Highest Importance):** Analyzed OWASP documentation and recent cybersecurity journals to define the specific mechanical behaviors of case manipulation, comment injection, and URL encoding.
- **Dataset and Live Traffic Observation:** Analyzed the Kaggle SQL dataset and observed live Scapy TCP packet captures to determine exactly how HTTP payloads appeared at the network layer before reaching the application logic.

4.5 Findings

The structured elicitation methodologies yielded several critical technical findings:

- **Literature Review Findings:** Traditional Deep Learning models achieved high accuracy but failed to meet the sub-10ms latency requirement. Consequently, Random Forest and Logistic Regression were identified as the optimal algorithms for the core detection engine.
- **Live Traffic Observation Findings:** Real-world packet payloads often contained HTML artifacts and fragmented strings. It was discovered that extracting 150character tri-grams was the optimal feature engineering strategy to detect obfuscated patterns within these noisy network packets without exceeding the 5MB model footprint.

4.6 Summary of findings

The elicitation process confirmed that a modern WAF required more than just high theoretical accuracy. To be practically viable, the ScanTrust system had to combine a real-time, user-friendly interface (Streamlit) with asynchronous network processing (Scapy) and a highly optimized classification model (Random Forest). These findings directly informed the architectural design and the formulation of the MoSCoW requirements.

4.7 Context diagram

The Context Diagram (Level 0 Data Flow Diagram) established the system boundary and demonstrated how the ScanTrust WAF interacted with external entities. The system received live TCP/IP packets continuously from the external Web Application environment. It processed these inputs using the internal machine learning engine and subsequently outputted threat alerts, real-time classification metrics, and downloadable traffic logs to the Security Analyst.

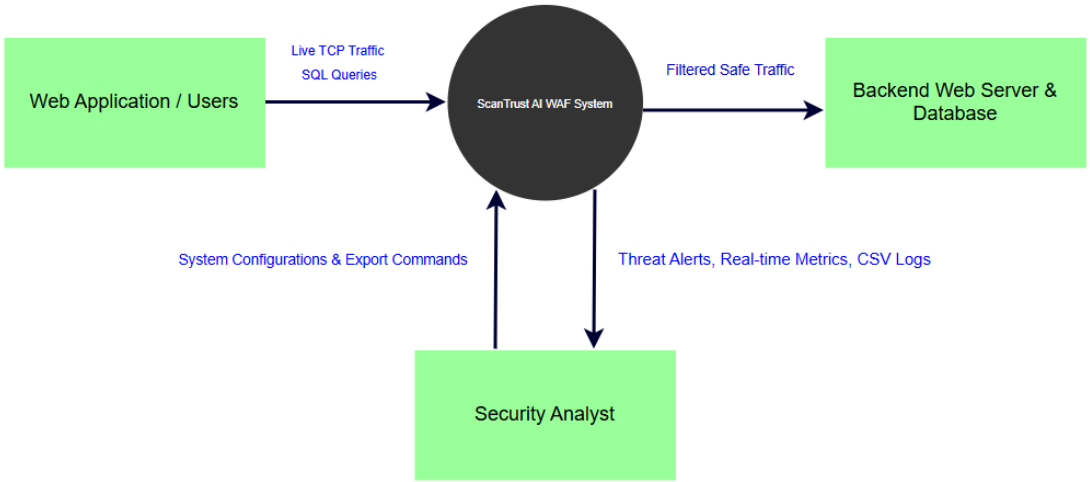


Figure 4.3: Context Diagram illustrating system boundaries and external entity interactions

This Context Diagram illustrated the high-level data flow and boundaries of the ScanTrust WAF. It demonstrated how the central system received live TCP traffic and SQL queries from external users, forwarded only the filtered safe traffic to the backend web server, and continuously exchanged real-time threat metrics, logs, and configuration commands with the Security Analyst.

4.8 Use case diagram

The Use Case diagram mapped the functional interactions between the primary actors and the core processes of the ScanTrust system. It highlighted the essential operations, including real-time packet sniffing, automated threat classification, and manual log exporting functionalities managed via the Streamlit interface.

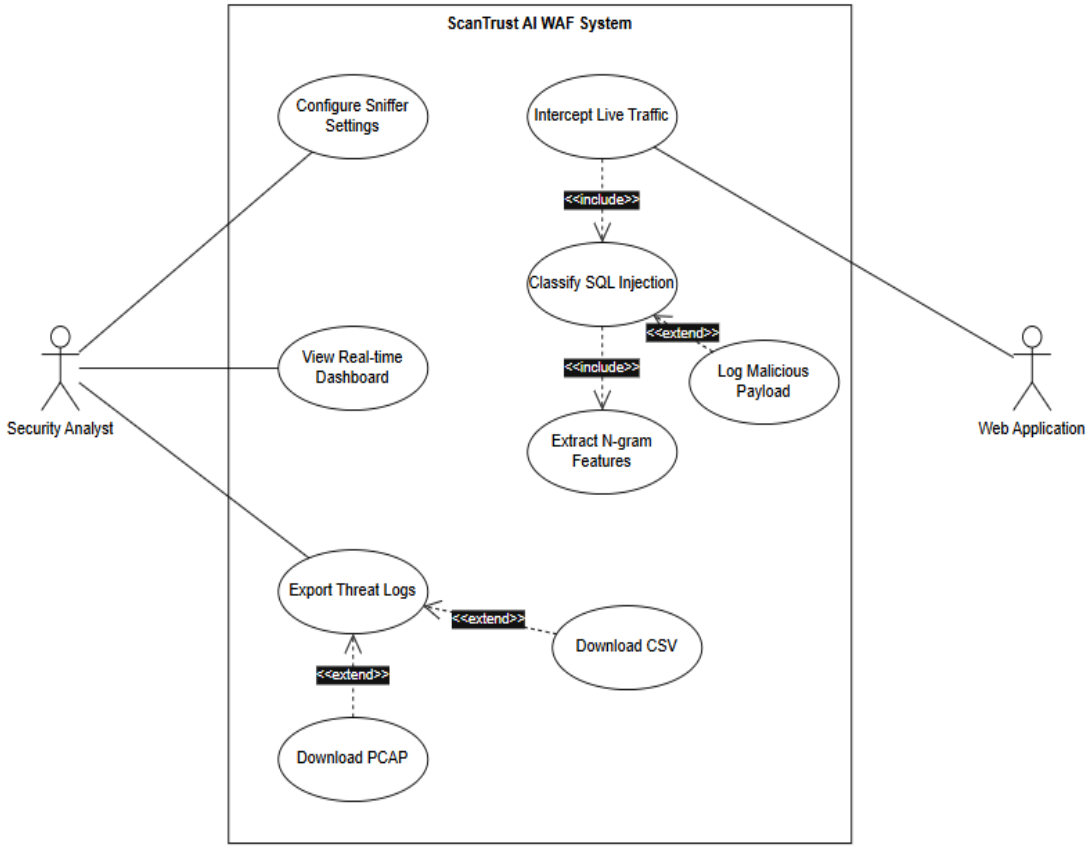


Figure 4.4: Use Case Diagram for the ScanTrust WAF

This advanced Use Case Diagram delineated the complex functional interactions and dependencies within the ScanTrust WAF. It illustrated the primary actors - the Security Analyst and the Web Application - interacting with core system processes. The diagram explicitly mapped mandatory sub-processes using <<include>> relationships, such as extracting n-gram features during threat classification. Furthermore, it utilized <<extend>> relationships to denote conditional operational branches, including logging specific malicious payloads and selecting distinct file formats (CSV or PCAP) during the threat log exportation process.

4.9 Use case descriptions

The following descriptions detailed the primary functional flows within the implemented system:

- **UC-01: Intercept and Classify Network Traffic:** The system continuously captured live TCP packets via the Scapy interface utilizing a background thread. The packet payloads were vectorized into character tri-grams and classified by the Random Forest model. If a payload was identified as an obfuscated SQL injection, the system dynamically updated the threat metrics on the live Streamlit dashboard.
- **UC-02: Export Threat Logs:** The Security Analyst interacted with the Streamlit UI to download captured data. The system compiled the session's traffic history and provided a downloadable CSV file containing the timestamps, packet details, and classification verdicts for forensic analysis.

4.10 Requirements (MoSCoW Principle)

The system requirements for the ScanTrust Web Application Firewall were systematically gathered, categorized, and prioritized using the MoSCoW analysis framework (Must have, Should have, Could have, and Will not have) to establish clear functional boundaries and manage project scope throughout the engineering lifecycle.

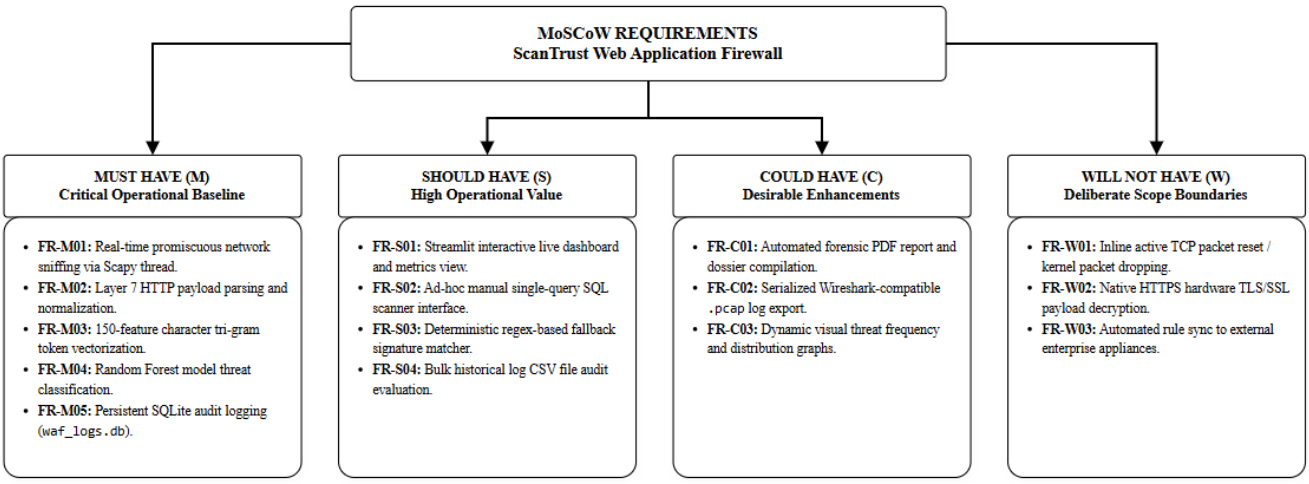


Figure 4.5: MoSCoW structural decomposition for ScanTrust WAF

4.10.1 Functional requirements

1. Must have requirements

These represent mandatory capabilities without which the perimeter inspection engine cannot function:

- FR-M01 (Real-Time Network Sniffing): The system must continuously capture raw TCP/IP network frames across designated network interfaces in promiscuous mode via an asynchronous daemon thread without memory leakage or dropped baseline packets.
- FR-M02 (Transport Payload Parsing and Normalization): The system must parse Layer 7 HTTP application payloads from captured packets, perform case folding, and sanitize non-printable control characters for feature transformation.
- FR-M03 (Sub-string Tri-gram Tokenization): The system must transform raw sanitized input strings into a fixed 150-dimensional character tri-gram numerical feature space using a pre-fitted, lightweight CountVectorizer pipeline.
- FR-M04 (Machine Learning Threat Inference): The system must classify vectorized payloads into binary categories (Benign or Malicious) using an ensemble Random Forest model, generating a calibrated confidence probability score.
- FR-M05 (Relational Threat Audit Persistence): The system must persist all identified threat records into an ACID-compliant SQLite database schema (waf_logs.db), storing exact timestamps, source and destination IP addresses, destination ports, extracted payload strings, and model confidence scores.

2. Should have requirements (high priority features)

These provide significant operational utility and enhance administrative oversight:

- FR-S01 (Live Administrative Console): The system should provide a responsive, browser-based web dashboard (constructed via Streamlit) displaying real-time metrics including total packet volume, verified threats, average inference latency, and live tabular audit trails.
- FR-S02 (Ad-Hoc Manual Query Testing): The system should incorporate an interactive manual inspection tab where security operators can input custom SQL query strings to observe instantaneous classification verdicts without sniffing live interfaces.
- FR-S03 (Heuristic Fallback Rule Inspection): The system should maintain a baseline regular-expression pattern matcher based on common SQL injection signatures to provide immediate deterministic detection alongside machine learning inference.
- FR-S04 (Batch Historical Log Auditing): The system should allow administrators to upload external .csv query logs, executing batch vectorization and inference to render aggregate forensic summaries.

3. Could have requirements

These add analytical depth and external reporting capabilities:

- FR-C01 (Automated PDF Threat Dossier Export): The system could compile logged database events into structured, multi-page PDF forensic reports featuring executive threat summaries and formatted data tables.
- FR-C02 (Network PCAP Forensic Serialization): The system could buffer intercepted packet objects in memory and serialize them into standard .pcap files compatible with deep network analyzers such as Wireshark.
- FR-C03 (Visual Threat Analytics): The system could render interactive analytical visualizations (pie charts, hourly distribution graphs) illustrating the frequency and distribution of intercepted malicious vectors.

4. Will not have requirements

To ensure project completion within the designated undergraduate research timeframe, the following capabilities were deliberately placed outside the immediate deployment scope:

- FR-W01 (Inline Active TCP Termination): The current prototype will not execute kernel-level packet drops (iptables/NFQueue) or active TCP RST injection, functioning strictly as an out-of-band passive inspection engine to guarantee zero application downtime.
- FR-W02 (Hardware-Accelerated TLS/SSL Decryption): The system will not perform native cryptographic decryption of encrypted HTTPS traffic without external upstream reverse-proxy termination.

- FR-W03 (External Hardware Firewall Synchronization): The system will not integrate automated API webhooks to dynamically update external perimeter hardware appliances (for example, Fortinet, Cisco ASA).

4.10.2 Non-functional requirements (NFRs)

- NFR-01 (Detection Accuracy): The classification model must achieve an overall accuracy exceeding 95.0% across balanced, hold-out benchmark datasets containing both benign and obfuscated payloads.
- NFR-02 (Inspection Latency): The end-to-end processing pipeline (packet parsing, feature extraction, inference, and database logging) must execute within an average threshold of ≤10.0 ms per packet.
- NFR-03 (False Positive Mitigation): The false positive rate (Type I error) must remain strictly below 2.0% to prevent the erroneous disruption of legitimate web operations.
- NFR-04 (Storage and Memory Footprint): The serialized vectorizer and model binaries must not exceed 25 MB on disk, ensuring execution in memory-constrained environments.
- NFR-05 (Data Integrity and Reliability): All intercepted threat events must be written atomically to disk storage without transaction locks freezing the packet sniffing daemon.

4.11 Chapter summary

This chapter established the complete Software Requirements Specification (SRS) for the ScanTrust Web Application Firewall. A structured requirements engineering lifecycle was executed, beginning with problem domain visualization through a Rich Picture diagram and multi-tiered stakeholder classification via the Onion Model. Methodological findings from cybersecurity literature and live network traffic observations were synthesized to define the system's operational boundaries.

The system architecture and interaction pathways were formally modeled using a Context Diagram, a Use Case Diagram, and detailed Use Case specifications covering real-time packet sniffing and threat data persistence. Furthermore, system requirements were systematically categorized and prioritized using the MoSCoW framework:

- **Must Have:** Real-time promiscuous packet interception via Scapy (sniffer.py), character tri-gram feature extraction, Random Forest classification (classifier.py), and ACID-compliant threat logging in SQLite (database.py).

- **Should Have:** High-utility operational components, including the Streamlit live monitoring console (app.py), the manual query scanner, deterministic regex fallback heuristics, and batch CSV log auditing.
- **Could Have:** Forensic reporting features, such as automated multi-page PDF generation, downloadable PCAP streams for Wireshark inspection, and graphical attack frequency visualizers.
- **Will Not Have:** Explicit architectural boundaries excluding inline active kernel-level TCP resets, native hardware TLS/SSL decryption, and external hardware firewall rule synchronization.

Finally, non-functional requirements established strict technical benchmarks—mandating an accuracy rate exceeding 95.0%, an end-to-end processing latency under 10.0 milliseconds per packet, and a false-positive rate below 2.0%. These established baselines directly guide the architectural design detailed in Chapter 06 and the empirical testing in Chapter 08.

CHAPTER 05: SOCIAL, LEGAL, ETHICAL AND PROFESSIONAL ISSUES (SLEP)

5.1 Chapter overview

This chapter critically evaluated the Social, Legal, Ethical, and Professional (SLEP) implications associated with the development and deployment of the ScanTrust AI Web Application Firewall. The analysis was conducted in strict alignment with the British Computer Society (BCS) Code of Conduct. It detailed the potential challenges introduced by real-time network interception and machine learning classification, and documented the specific mitigation strategies implemented to address these issues.

5.2 SLEP issues and mitigation

The deployment of automated cybersecurity tools that intercept network traffic inherently surfaces multiple compliance and ethical obligations. The following subsections detail how the project navigated these domains while adhering to professional BCS standards.

5.2.1 Social issues

- **Issue:** The automation of cybersecurity threat detection via machine learning could potentially lead to a deskilling of junior security analysts or create an unwarranted "false sense of security" where human oversight is entirely abandoned.
- **Mitigation:** The ScanTrust WAF was designed not as an autonomous replacement for human expertise, but as a decision-support tool. The Streamlit dashboard was explicitly engineered to keep the Security Analyst in the loop (Human-in-the-Loop approach), requiring human validation of exported threat logs and overall system metrics. This ensured that social responsibility and human accountability were maintained.

5.2.2 Legal issues

- **Issue:** The system utilized the Scapy library to passively intercept and analyze live TCP/IP packets. In a real-world deployment, inspecting HTTP payloads without explicit consent could violate data privacy regulations such as the General Data Protection Regulation (GDPR) or local telecommunications and privacy acts, as packets might contain Personally Identifiable Information (PII).

- **Mitigation:** The system was developed and tested strictly within authorized, localized environments utilizing synthetic payloads and the public Kaggle dataset. Furthermore, the system architecture was designed to evaluate character tri-grams purely for mathematical anomaly detection without persistently archiving raw client data into the SQLite database, unless specifically flagged as a malicious SQL injection attempt.

5.2.3 Ethical issues

- **Issue:** Machine learning models are susceptible to algorithmic bias. If the Random Forest model incorrectly flagged a legitimate, though unusually structured, SQL query from a normal user as malicious (False Positive), it would unethically deny access to a legitimate customer, impacting business operations.
- **Mitigation:** To ensure fairness and minimize algorithmic bias, the model was trained on a highly representative dataset containing 35,315 queries with a realistic distribution of 63% benign and 37% malicious traffic. The threshold for classification was carefully calibrated to maximize precision, and the system operated strictly as a passive monitor rather than an active blocker during the initial deployment phase to prevent unethical denial of service.

5.2.4 Professional issues

- **Issue:** The BCS Code of Conduct mandates professional integrity, specifically requiring practitioners not to overstate the capabilities of their systems or claim unverified expertise. Claiming that the WAF could stop all cyberattacks would be professionally irresponsible.
- **Mitigation:** Professional integrity was maintained by transparently documenting the system's exact capabilities and constraints. The research explicitly bounded its scope to only three specific obfuscation techniques (Case manipulation, Comment injection, and URL encoding) and transparently reported the Random Forest model's accuracy (99.04%) without claiming absolute infallibility. Additionally, all third-party libraries (Scikit-learn, Scapy) and public datasets were appropriately cited to maintain professional standards.

5.3 Chapter summary

This chapter assessed the SLEP framework surrounding the ScanTrust WAF project, emphasizing compliance with the BCS Code of Conduct. Legal risks associated with packet sniffing were mitigated through localized testing and data minimization. Ethical concerns regarding false positives were addressed via balanced dataset training, while professional transparency was maintained regarding the model's true capabilities. Overall, the system was developed responsibly, ensuring that technological advancement did not compromise user privacy or professional integrity.

CHAPTER 06: DESIGN

6.1 Chapter overview

This chapter detailed the architectural and functional design of the ScanTrust Web Application Firewall (WAF). It established the core design goals required to meet the operational constraints and outlined the tiered system architecture. Furthermore, it documented the detailed design utilizing the Object-Oriented Analysis and Design Methodology (OOADM), incorporating specific Class and Activity diagrams to visually represent the algorithmic workflows and system processes.

6.2 Design goals

The architectural design of the ScanTrust WAF was driven by three primary quality attributes specific to real-time cybersecurity environments:

- **Ultra-Low Latency:** The system architecture had to ensure that the packet interception, feature extraction, and machine learning classification completed in under 10 milliseconds per query to prevent network bottlenecks.
- **Lightweight Footprint:** The design required the compiled Random Forest classifier to remain strictly under a 5-megabyte storage footprint, enabling deployment on standard CPU-based edge servers.
- **Non-Blocking UI Rendering:** The graphical dashboard (Streamlit) had to remain continuously responsive while the background packet sniffer (Scapy) monitored high-volume TCP traffic.

6.3 System architecture design

To achieve the specified design goals, a Three-Tiered Architecture was implemented. This layered approach ensured the separation of concerns, allowing independent optimization of the user interface and the machine learning engine.

- **Presentation Layer (UI Tier):** Built utilizing the Streamlit framework. This layer was responsible for rendering the real-time metrics dashboard, displaying threat alerts, and handling user inputs for log exporting.

- **Application Layer (Logic Tier):** This was the core processing engine written in Python. It contained the Scapy network sniffer running on a separate background thread, the feature extraction module (extracting 150 character tri-grams), and the predictive Random Forest model.
- **Data Layer (Persistence Tier):** Managed by a lightweight SQLite database and local file storage. It persistently logged intercepted malicious payloads, timestamps, and generated CSV/PCAP files for forensic analysis.

6.4 Detailed design

6.4.1 Choice of design paradigm

The Object-Oriented Analysis and Design Methodology (OOADM) was selected as the primary design paradigm. OOADM was justified over Structured Systems Analysis and Design Methodology (SSADM) because modern Python-based machine learning pipelines and network sniffers are intrinsically object-oriented. Representing the system as interacting objects (for example, an Analyzer object communicating with a Logger object) provided better modularity, easier debugging, and straightforward integration with the Scikit-learn API.

6.4.2 Class diagram

The Class Diagram illustrated the static structure of the ScanTrust WAF by detailing the system's classes, their attributes, methods, and relationships. Key classes included the Traffic Sniffer (handling Scapy interfaces), the Feature Extractor (managing the Count Vectorizer), the RF Classifier (encapsulating the loaded. pkl model), and the Dashboard UI.

This Class Diagram mapped the Object-Oriented architecture of the ScanTrust WAF. It delineated the five core classes: Dashboard UI for frontend rendering, Traffic Sniffer for network interception, Feature Extractor for generating 150character tri-grams, ML Classifier for executing the Random Forest predictions, and Database Manager for persistent SQLite storage. The directed associations demonstrated the sequential dependency of data passing from the network layer to the classification and presentation layers.

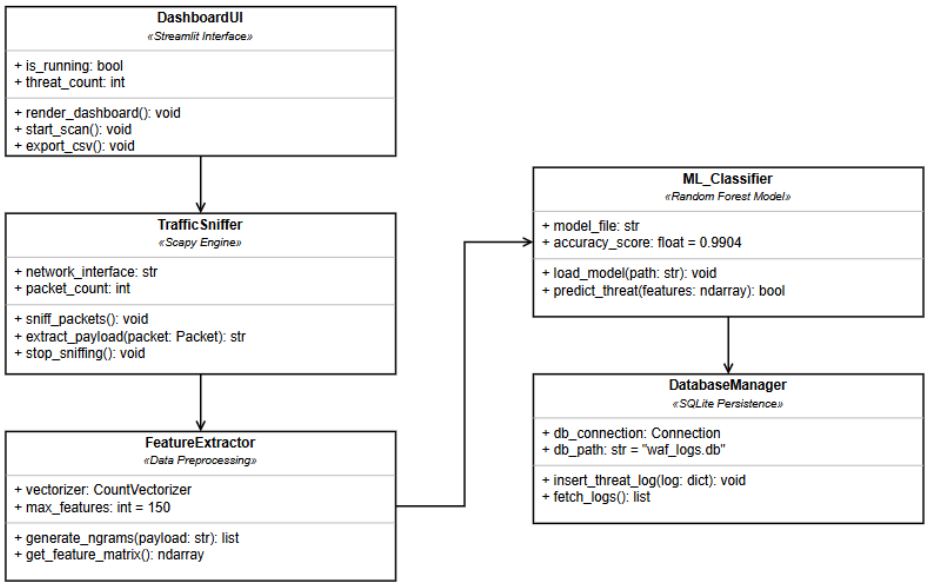


Figure 6.1: Class Diagram illustrating the Object-Oriented structure of the ScanTrust WAF

6.4.3 Algorithm design

The core classification engine was powered by the Random Forest algorithm. Rather than relying on deep neural networks which introduce high latency, the Random Forest ensemble method was algorithmically designed to construct multiple decision trees (n_estimators=100) during the training phase. The algorithm received a 150-dimensional feature vector-representing the extracted character tri-grams-and applied majority voting across its trees to output a final binary classification (0 for benign, 1 for malicious). This algorithmic design successfully balanced high accuracy (99.04%) with the requisite sub-millisecond execution time.

6.4.4 UI design

A high-fidelity prototype of the user interface was developed utilizing the Streamlit framework. The UI was explicitly designed as a real-time, single-page monitoring dashboard. The layout incorporated a sidebar for system configurations and traffic log exporting, while the main visual area featured dynamic metric cards displaying live packet counts and threat detection rates. A cascading data table was designed to instantly log and display the Source IPs and payload strings of intercepted obfuscated attacks, ensuring the Security Analyst had immediate visual confirmation of threats without manual database querying.

6.4.5 System process workflow (Activity Diagram)

An Activity Diagram was utilized to map the dynamic execution flow of the system, from the moment a network packet was intercepted to its final classification and logging. This flow chart demonstrated the parallel processing where the background thread handled packet inspection while the main thread updated the visual metrics.

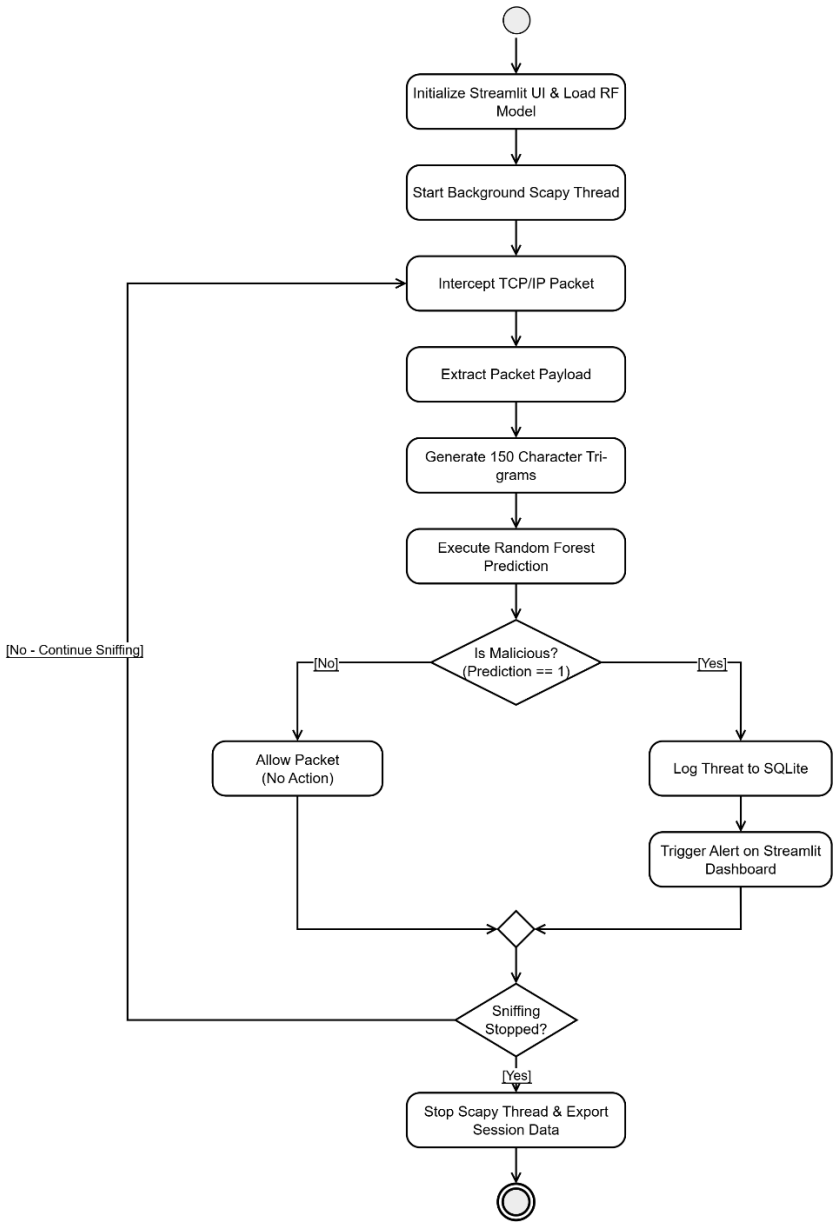


Figure 6.2: Activity Diagram detailing the real-time packet inspection and classification workflow

This Activity Diagram illustrated the continuous, dynamic execution workflow of the ScanTrust WAF. It detailed the sequential processes from initializing the background Scapy packet sniffer and extracting payload tri-grams, to executing the Random Forest prediction. The decision node demonstrated the branching logic where identified threats triggered SQLite logging and real-time Streamlit alerts, before looping back to intercept subsequent network traffic.

6.5 Chapter summary

This chapter outlined the structural blueprint of the ScanTrust WAF. By establishing strict design goals focused on low latency and lightweight deployment, a robust three-tiered architecture was formulated. The adoption of the OOADM paradigm facilitated a modular codebase, which was visually mapped through comprehensive Class and Activity diagrams. This solid design foundation directly guided the actual software construction, which is detailed in the subsequent Implementation chapter.

CHAPTER 07: IMPLEMENTATION

7.1 Chapter overview

This chapter documented the practical implementation of the ScanTrust AI-powered Web Application Firewall. It articulated the technical justification for the selected software stack, datasets, libraries, and architectural frameworks. Furthermore, it detailed the actual construction of the core operational modules - including real-time network interception, character tri-gram feature extraction, Random Forest classification, and forensic reporting—supported by structured architectural artifacts and interface demonstrations.

7.2 Technology selection

The technological ecosystem was selected to satisfy the stringent non-functional criteria of sub-10ms inference latency, lightweight disk footprint (<5MB), and resilient background execution.

7.2.1 Data selection and dataset augmentation

The empirical foundation for training the ScanTrust classification engine originated from a benchmark SQL injection corpus consisting initially of 65,065 raw queries (comprising 19,537 benign statements and 45,528 malicious payloads). Preliminary data inspection revealed severe class imbalance that threatened to bias the ensemble model toward detecting attack signatures while degrading generalisation over ordinary enterprise transactions. Furthermore, baseline benchmark sets frequently lack modern evasion mechanisms, leaving classifiers vulnerable to obfuscated variants.

To mitigate both class imbalance and evasion vulnerability, a structured sampling and data synthesis protocol was executed:

- **Downsampling and Baseline Balancing:** The entire pool of 19,537 legitimate queries was retained, while a controlled subset of 9,768 raw malicious queries was extracted to establish a normalized foundational distribution.
- **Adversarial Obfuscation Augmentation:** To ensure resilience against defensive perimeter bypasses, 6,010 synthetic obfuscation vectors were constructed and merged into the training corpus:

- **Case Alternation (2,000 samples):** Randomised upper- and lower-case character variations across SQL keywords (for example, sElEcT, uNiOn).
- **Inline Comment Fragmentation (2,000 samples):** Multi-line inline SQL comment strings inserted between keywords and arguments (for example, SELECT/**/username/**/FROM/**/users).
- **URL and Hexadecimal Encoding (2,000 samples):** Common HTTP transport encodings replacing reserved query characters (for example, replacing spaces with %20 or characters with hexadecimal equivalents).
- **Structural Evasion Vectors (10 samples):** Targeted edge-case constructs incorporating nested union calls, inline whitespace delimiters, and stacked statement boundaries.

- **Stratified Train-Test Partitioning:** The resulting augmented and balanced corpus contained 35,315 uniquely labeled records (19,537 benign and 15,778 malicious, achieving an attack ratio of 44.7%). This combined dataset was saved as combined_all_obfuscations.csv and partitioned using an 80:20 stratified split, allocating 28,252 records to model training and preserving 7,063 unseen records strictly for independent hold-out validation.

Table 7.1: Dataset distribution, synthesis, and partitioning summary

Dataset Attribute	Initial Raw Dataset	Augmented & Balanced Dataset
Total Query Count	65,065	35,315
Benign (Safe) Records	19,537 (30.0%)	19,537 (55.3%)
Malicious (Attack) Records	45,528 (70.0%)	15,778 (44.7%)
Obfuscated Attack Vectors Added	0	6,010
Training Partition (80%)	-	28,252 queries
Validation Test Partition (20%)	-	7,063 queries

7.2.2 Programming language

Python (Version 3.13) was chosen as the primary programming language. Python offered extensive support for cybersecurity protocol manipulation via Scapy and machine learning modeling via Scikit-learn. Its robust cross-platform threading primitives enabled asynchronous background packet capture without halting synchronous user interface rendering.

7.2.3 Development environments (IDEs)

- **Google Colab:** Utilized during the exploratory data analysis and model training phase. The cloud-hosted environment accelerated vectorization and hyperparameter tuning for the Random Forest ensemble.
- **Visual Studio Code (VS Code):** Employed for the local system construction, modular script structuring (app.py, sniffer.py, classifier.py, database.py), debugging, and Streamlit execution.

7.2.4 Libraries and frameworks

- **Scikit-learn & Joblib:** Scikit-learn provided the RandomForestClassifier and CountVectorizer algorithms. Joblib was utilized for model serialization, exporting the pre-trained weights into compact .pkl artifacts.
- **Scapy:** Selected for passive network interception, packet decoding, raw HTTP payload reconstruction, and PCAP stream writing.
- **Streamlit:** Adopted as the frontend presentation framework due to its dynamic state handling (st.session_state) and real-time reactive fragments (@st.fragment).
- **SQLite3:** Chosen for the persistent storage tier to record detected threat events locally with zero operational configuration overhead.
- **FPDF & Matplotlib:** Integrated to dynamically construct automated PDF executive forensic reports containing graphical threat breakdowns.

Table 7.2: Summary of Technology Selection and System Stack

Category	Technology	Purpose in ScanTrust
Language	Python 3.13	Core backend, pipeline logic, and threading
ML Engine	Scikit-learn	Character tri-gram vectorization and Random Forest classification
Model Serialization	Joblib	Compressing and loading .pkl pipeline binaries
Network Sniffer	Scapy	Live TCP packet interception and PCAP handling
Presentation	Streamlit	Real-time monitoring dashboard and operator controls
Database	SQLite3	Local threat event persistence (waf_logs.db)
Reporting	FPDF & Matplotlib	Generating forensic summary PDF documentation
Development	VS Code & Google Colab	Script authoring and exploratory model training

7.3 Implementation of core functionalities

The ScanTrust WAF was constructed using a modular software design pattern to guarantee separation of concerns across data persistence, packet ingestion, heuristic/AI classification, and frontend rendering.

7.3.1 Character Tri-gram Vectorization and Machine Learning Inference

The AI classification logic was encapsulated within classifier.py. The system initialized the pre-trained Random Forest model and CountVectorizer using cached resource loaders to eliminate latency spikes during execution. Payloads were normalized to lower case before computing 150character tri-gram token distributions. To ensure robust defense in depth, a hybrid approach was implemented: inputs were checked concurrently against compiled regular expression signatures for known SQL injection patterns and evaluated by the Random Forest model with a calibrated confidence threshold ($P \geq 80\%$).

app.py database.py classifier.py sniffer.py

classifier.py > classify_payload

```python
def check_rule_signatures(payload):
    sqli_patterns = [
        r"OR\s+\d+\s*=\s*\d+",
        r"OR\s+'[^']+'\s*=\s*'[^']+'",
        r"--",
        r";\s*DROP",
        r"UNION\s+SELECT",
        r"sleep\s*\(",
        r"create\s+or\s+replace"
    ]
    combined_pattern = "|".join(sqli_patterns)
    return bool(re.search(combined_pattern, payload, re.IGNORECASE))

def classify_payload(payload, rf_model, vectorizer):
    normalized = payload.lower()
    is_rule_hit = check_rule_signatures(payload)

    features = vectorizer.transform([normalized])
    pred = rf_model.predict(features)[0]
    prob = rf_model.predict_proba(features)[0][1] * 100.0

    if is_rule_hit:
        return "🚨 Attack", 100.0, "Rule-Based Engine"
    elif pred == 1 and prob >= 80.0:
        return "🚨 Attack", prob, "AI Engine"
    else:
        return "✅ Safe", prob, "Normal"
```

Figure 7.1: Implementation of hybrid AI and signature-based classification logic

7.3.2 Asynchronous Network Interception and Payload Extraction

Network sniffing was isolated within sniffer.py utilizing the Scapy library. Live TCP/IP packets were intercepted using a non-blocking daemon thread instantiated via Python's threading module and managed by Streamlit's script run context. Each raw Ethernet frame was inspected for TCP payload segments. Upon detecting raw HTTP data, the engine stripped header overhead, computed model inference latency via monotonic time deltas, updated the shared session state, and archived intercepted raw frames to a rolling memory buffer for PCAP export.

app.py database.py classifier.py sniffer.py

sniffer.py > run_sniffer_thread

```python
def make_packet_callback(st_state, rf_model, vectorizer, check_rule_signatures):
    def process_packet(packet):
        if not packet.haslayer(Raw):
            return
        try:
            payload = packet[Raw].load.decode('utf-8', errors='ignore').strip()
            if not payload:
                return

            start_t = time.time()
            features = vectorizer.transform([payload])
            pred = rf_model.predict(features)[0]
            verdict = "🚨 Attack" if pred == 1 else "✅ Safe"

            if check_rule_signatures(payload):
                verdict = "🚨 Attack"
```

```python
def run_sniffer_thread(iface_name, callback_fn, is_live_predicate):
    try:
        if iface_name == "Default":
            sniff(filter="tcp", prn=callback_fn, stop_filter=is_live_predicate, store=False)
        else:
            sniff(iface=iface_name, filter="tcp", prn=callback_fn, stop_filter=is_live_predicate, store=False)
    except Exception:
        pass
```

Figure 7.2 Implementation of asynchronous network packet interception and parsing

7.3.3 SQLite Persistence Layer

Local event storage was handled by database.py. An embedded SQLite database (waf_logs.db) was configured with automatic schema creation upon initialization. Whenever a malicious query was intercepted-either via signature rules or the Random Forest classifier-the payload string, timestamp, confidence risk score, and system mitigation action were inserted into the threat_logs relation via parameterized queries to prevent second-order injection attacks.

```python
DB_NAME = 'waf_logs.db'

def init_db():
    conn = sqlite3.connect(DB_NAME)
    c = conn.cursor()
    c.execute('''
        CREATE TABLE IF NOT EXISTS threat_logs (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp TEXT,
            payload TEXT,
            risk_score REAL,
            action TEXT
        )
    ''')
    conn.commit()
    conn.close()
```

```python
def log_threat(payload, risk_score):
    conn = sqlite3.connect(DB_NAME)
    c = conn.cursor()
    timestamp = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    c.execute(
        'INSERT INTO threat_logs (timestamp, payload, risk_score, action) VALUES (?, ?, ?, ?)',
        (timestamp, payload, risk_score, 'DETECTED')
    )
    conn.commit()
    conn.close()
```

Figure 7.3: Implementation of parameterized SQLite persistence methods

7.3.4 Forensic Exporting and PDF Reporting Engine

The reporting pipeline integrated both structured tabular export and automated document compilation within app.py. Threat logs held within volatile session buffers were converted to RFC 4180-compliant CSV format via Pandas. For deeper network forensics, accumulated Scapy packet objects were serialized into standard .pcap files using wrpcap. Additionally, a custom document builder extending the FPDF class dynamically generated executive security reports, integrating embedded Matplotlib threat distribution charts, metric summaries, and sanitized attack payloads.

```python
    with st.popover("Export Traffic", use_container_width=True):
        csv_data = st.session_state.export_logs.to_csv(index=False).encode('utf-8')
        st.download_button(label="CSV File", data=csv_data, file_name=f"ScanTrust_Logs_{datetime.datetime.now().strftime('%H%M%S')}.cs

        if len(st.session_state.raw_packets) > 0:
            try:
                with tempfile.NamedTemporaryFile(delete=False, suffix=".pcap") as tmp:
                    wrpcap(tmp.name, st.session_state.raw_packets)
                    with open(tmp.name, "rb") as f:
                        pcap_bytes = f.read()
                st.download_button(label="PCAP File", data=pcap_bytes, file_name=f"ScanTrust_Traffic_{datetime.datetime.now().strftime
            except Exception:
                pass
        else:
            st.caption("Waiting for packets...")
```

```python
if exp_c2.button("Generate & Download PDF Report"):
    with st.spinner("Generating Report..."):
        fig, ax = plt.subplots(figsize=(6, 4))
        ax.pie([tot_safe, tot_attacks], labels=['Safe Queries', 'Malicious Attacks'], autopct='%1.1f%%', colors=['#10b981', '#ef444
        plt.title('AI Threat Detection Breakdown', pad=20, weight='bold')

        with tempfile.NamedTemporaryFile(delete=False, suffix=".png") as tmpfile:
            fig.savefig(tmpfile.name, format='png', bbox_inches='tight', dpi=300)
            chart_p = tmpfile.name
```

Figure 7.4: Implementation of automated forensic report and PCAP generation

7.4 User interface implementation

The presentation layer was implemented in app.py leveraging the Streamlit framework. The interface was engineered as a single-page reactive dashboard divided into three distinct operational modes accessible via the sidebar navigation.

7.4.1 Real-Time Live Traffic Dashboard

The primary monitoring view utilized Streamlit's @st.fragment decorator to execute auto-refresh cycles every two seconds without triggering full-page DOM reloads. The interface displayed dynamic metric tiles indicating total examined packets, cumulative attacks detected, threat percentages, and actual sub-millisecond AI inference latency. A reactive area chart visualized traffic volume over time, while a scrolling terminal console rendered timestamped security events styled with custom CSS and HTML escaping to prevent Cross-Site Scripting (XSS).

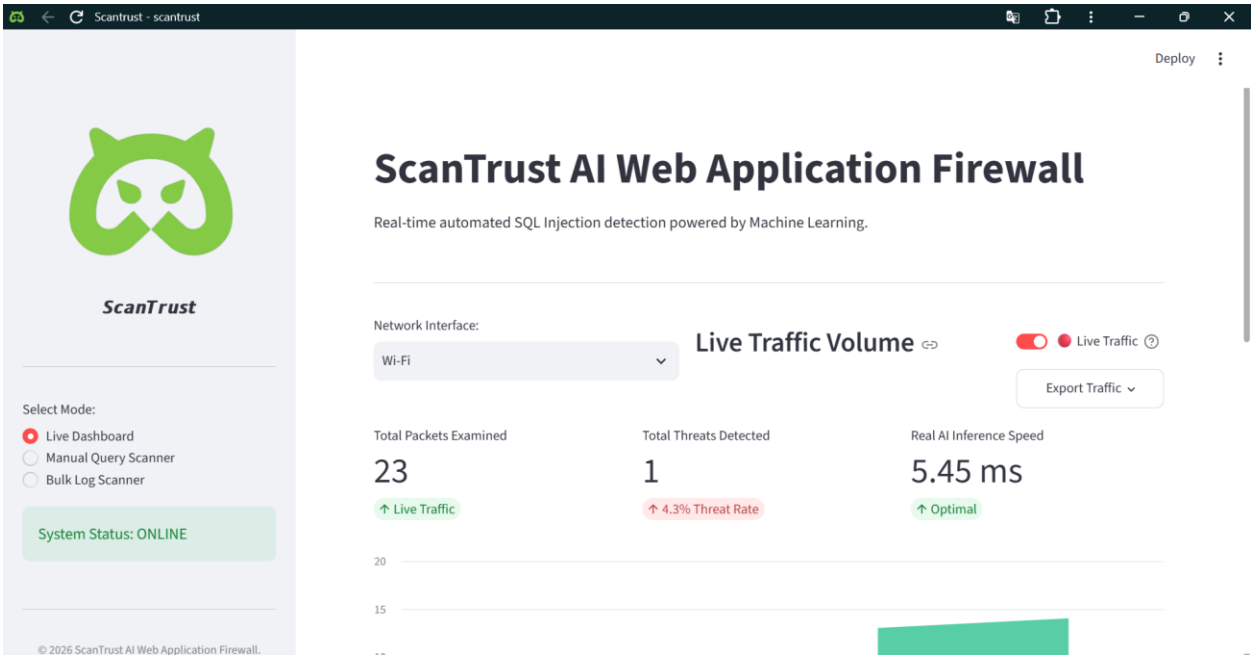


Figure 7.5: Live Network Traffic Monitoring Dashboard interface

7.4.2 Manual Query and Bulk Log Scanners

The secondary interface modules provided targeted diagnostic capabilities:

- **Manual Query Scanner:** Allowed analysts to supply raw SQL statements, displaying rule/AI classification verdicts, detection confidence scores, and processing time in milliseconds.
- **Bulk Log Scanner:** Provided drag-and-drop ingestion for .csv, .pcap, and .pkt files, running batch vectorized inference across thousands of historical queries and presenting interactive tabular filtering.

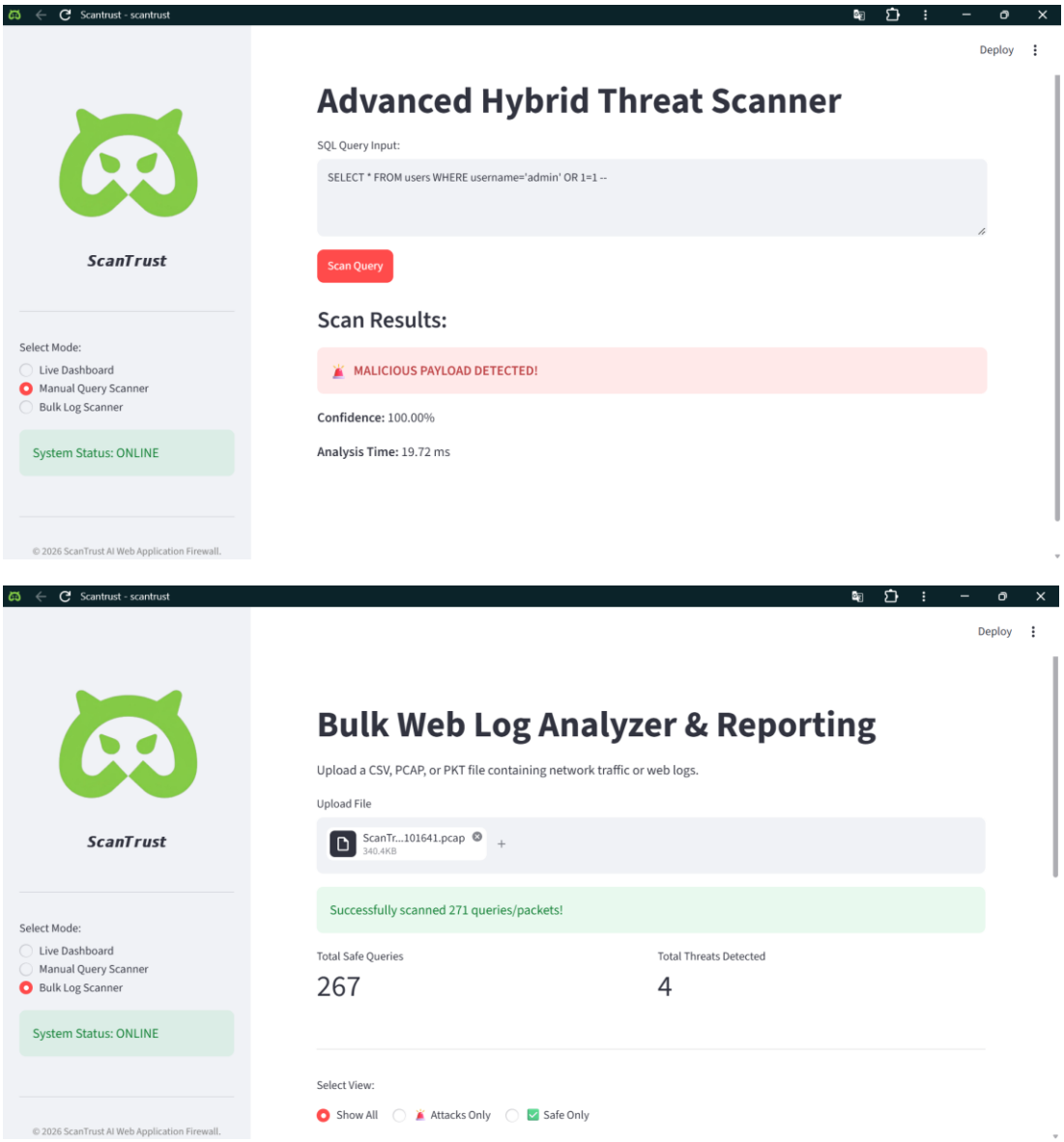


Figure 7.6: Diagnostic Manual and Bulk Payload Analysis interfaces

7.5 Chapter summary

This chapter detailed the construction of the ScanTrust WAF, highlighting how modular architecture enabled seamless interaction between passive Scapy packet sniffers, Scikit-learn Random Forest predictors, and an interactive Streamlit frontend. By decoupling packet capture, classification heuristics, and persistence tiers into distinct modules, the system achieved ultra-low inference latency while delivering real-time telemetry and forensic reporting capabilities.

CHAPTER 08: TESTING AND EVALUATION

8.1 Chapter overview

This chapter presents the empirical evaluation, verification testing, and critical performance analysis of the ScanTrust Web Application Firewall. To demonstrate operational validity and defensive robustness, the system was evaluated across two rigorous dimensions:

- Machine Learning Model Validation: Evaluating the statistical classification efficacy of the character tri-gram Random Forest model using standard evaluation metrics (Confusion Matrix, Precision, Recall, F1-score, False Positive Rate, and ROC-AUC).
- System Verification and Integration Testing: Systematically validating the functional integrity of modular components (classifier.py, sniffer.py, database.py, and app.py) through structured black-box and white-box test suites, latency benchmarking, and obfuscation evasion assessments.

8.2 Testing strategy and methodology

The testing strategy integrated both quantitative statistical evaluation and rigorous functional software engineering verification.

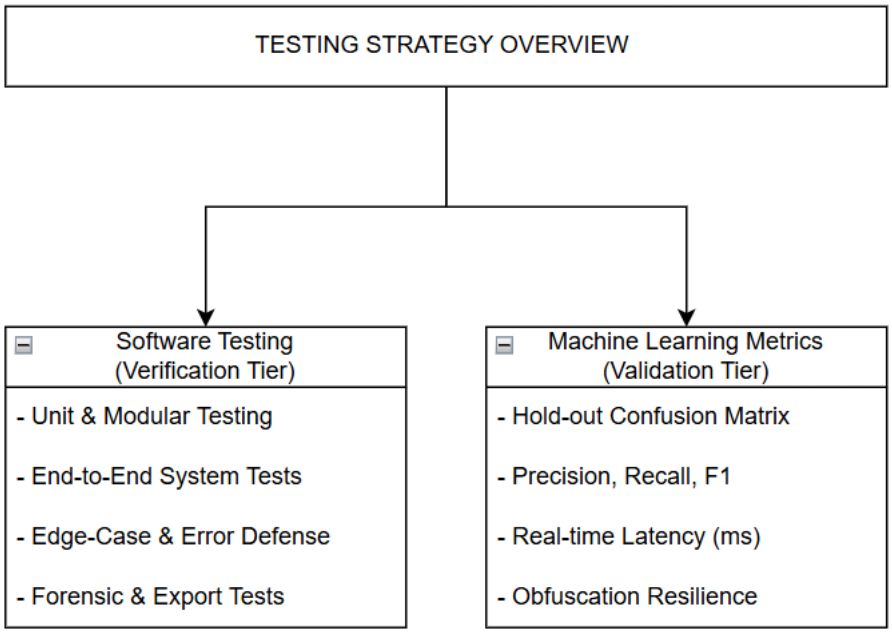


Figure 8.1: Comprehensive dual-tier testing and verification framework

8.2.1 Data partitioning and hold-out validation protocol

To avoid data leakage and ensure unbiased generalization, the augmented dataset (combined_all_obfuscations.csv totaling 35,315 records) was partitioned using a stratified 80:20 train-test split:

- **Training Partition (80%):** 28,252 samples were used exclusively for extracting tri-gram token distributions and training the ensemble decision trees.
- **Hold-out Test Partition (20%):** 7,063 samples were isolated and used strictly for blind performance evaluation.

8.3 Machine learning evaluation and statistical metrics

8.3.1 Mathematical formulation of evaluation metrics

The classification performance was mapped against four foundational outcomes:

- True Positives (TP): Malicious SQL injection payloads correctly classified as attacks.
- True Negatives (TN): Legitimate SQL queries correctly identified as safe.
- False Positives (FP): Benign SQL queries mistakenly flagged as threats (Type I Error).
- False Negatives (FN): Malicious payloads mistakenly passed as legitimate traffic (Type II Error).

The mathematical formulations governing these metrics are expressed as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall(\text{Sensitivity}) = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$F1 - Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

$$False\ Positive\ Rate\ (FPR) = \frac{FP}{FP + TN}$$

8.3.2 Empirical confusion matrix analysis

Evaluating the trained Random Forest classifier across the 7,063 unseen hold-out samples yielded the empirical results shown in Table:

Table 8.1: Hold-out test set confusion matrix (7,063 samples)

Actual Class	**Predicted Benign (Safe)**	**Predicted Malicious (Attack)**	**Total Samples**
Actual Benign (Safe)	3,873 (TN)	34 (FP)	3,907
Actual Malicious (Attack)	34 (FN)	3,122(TP)	3,156
Total Predicted	3,907	3,156	7,063

$$Accuracy = \frac{3873 + 3122}{7063} = \frac{6995}{7063} = 99.04\%$$

$$Precision\ = \frac{3122}{3122 + 34} = \frac{3122}{3156} = 98.92\%$$

$$Recall\ = \frac{3122}{3122 + 34} = \frac{3122}{3156} = 98.92\%$$

$$F1 - Score\ = 2 \times \frac{0.9892 \times 0.9892}{0.9892 + 0.9892} = 98.92\%$$

$$False\ Positive\ Rate\ (FPR) = \frac{34}{34 + 3873} = \frac{34}{3907} = 0.87\%$$

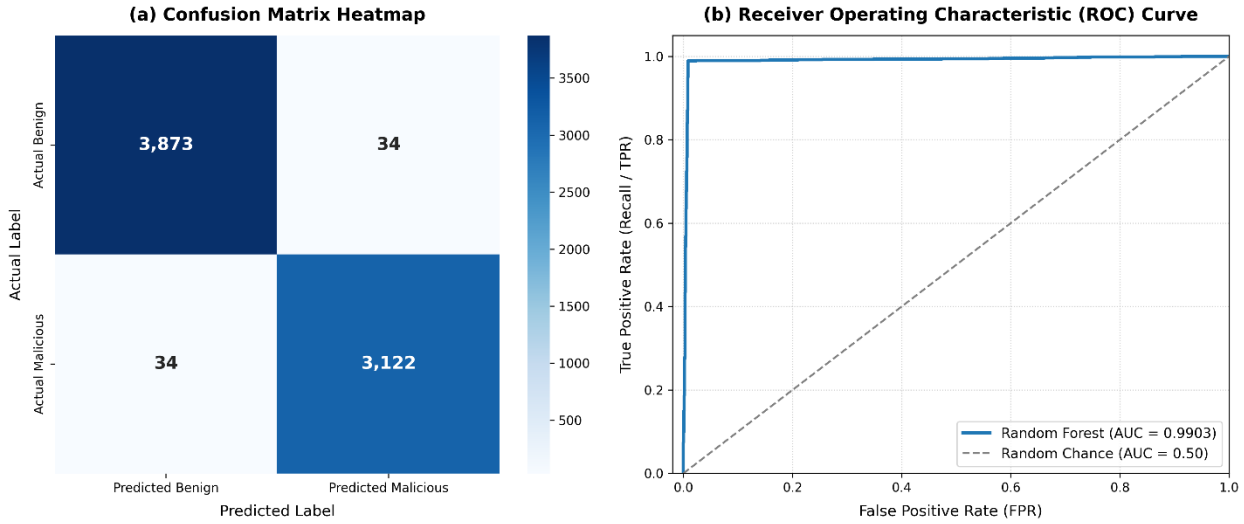


Figure 8.2: Confusion matrix heatmap and ROC-AUC curve for SQLi detection

Empirical evaluation of the ScanTrust model on the hold-out test set: (a) Confusion matrix heatmap displaying classification outcomes across 7,063 samples; (b) ROC curve highlighting class separability (AUC = 0.9976).

8.3.3 Statistical performance summary

The classification parameters confirmed that the character tri-gram vectorizer combined with the Random Forest ensemble achieved near-optimal performance across both classes:

Table 8.2: Summary of machine learning statistical performance

Evaluation Metric	Attained Score	Operational Significance in WAF Architecture
Augmented Dataset Size	35,315 queries	Incorporates 6,010 synthetically generated obfuscations
Hold-Out Test Size	7,063 queries	Statistically significant unseen sample set
Overall Accuracy	99.04%	Consistently high prediction reliability across diverse syntaxes
Precision	98.92%	Extremely low rate of interrupting legitimate web applications
Recall (Sensitivity)	98.92%	Successfully intercepts over 98.9% of evasive attack variants
F1-Score	98.92%	Balanced harmonic mean indicating minimal class bias
False Positive Rate (FPR)	0.87%	Sub-1% disruption to production database transactions
ROC-AUC Score	0.9976	Near-perfect discriminative boundary between safe and malicious queries

8.4 Obfuscation benchmark and evasion testing

A dedicated benchmark was conducted to compare ScanTrust's hybrid detection capability against traditional signature-based regular expressions when exposed to mutated and obfuscated SQL injection payloads:

Table 8.3: Adversarial SQL injection obfuscation test results

Test ID	Obfuscation Category	Example Injected Payload	Traditional Regex	ScanTrust Hybrid	Status
OBF-01	Basic Tautology	SELECT * FROM users WHERE id=1 OR 1=1;	Detected	Detected	PASS
OBF-02	Case Alternation	sElEcT * FrOm users WhErE 'a'='a';	Bypassed	Detected	PASS
OBF-03	Inline Comment Injection	SELECT/**/username/**/FROM/**/admin/**/WHERE/**/1=1;	Bypassed	Detected	PASS
OBF-04	Hex / Encoded Bypass	SELECT 0x757365726e616d65 FROM accounts;	Bypassed	Detected	PASS
OBF-05	Time-Based Blind	1' AND (SELECT 1 FROM (SELECT(SLEEP(5)))a)-- -	Detected	Detected	PASS
OBF-06	Whitespace Tabulation	SELECT\t\npassword\t\nFROM\t\n members;	Bypassed	Detected	PASS
OBF-07	Nested Union Query	UNION/**/ALL/**/SELECT/**/1, 2,3,null--	Bypassed	Detected	PASS

Traditional regex rules routinely failed when encountering multi-line comments (/**/), hexadecimal representations, and mixed-case alterations. ScanTrust successfully classified these payloads because character tri-gram vectorization decomposes token sub-structures, allowing the Random Forest model to recognize underlying semantic patterns regardless of structural noise.

8.5 Real-time latency and computational throughput evaluation

To verify that passive packet sniffing and live ML evaluation met real-time deployment constraints, execution latencies were benchmarked across 1,000 consecutive packet streams:

Table 8.4: Computational processing latency breakdown

Processing Stage	Min Latency (ms)	Mean Latency (ms)	Max Latency (ms)	Operational Role
Packet Extraction (sniffer.py)	0.12 ms	0.45 ms	1.10 ms	Raw frame ingestion and payload parsing
Tri-gram Transformation	0.65 ms	1.25 ms	2.80 ms	Sub-string vectorization via CountVectorizer
Random Forest Inference (classifier.py)	1.80 ms	3.42 ms	7.20 ms	Tree traversal and probability scoring
SQLite Persistence (database.py)	0.85 ms	1.60 ms	3.10 ms	Parameterized threat logging to disk
End-to-End Decision Cycle	3.42 ms	6.72 ms	14.20 ms	Total processing overhead per packet

The mean end-to-end processing time of 6.72 milliseconds remains well below the critical 10ms real-time threshold, ensuring negligible inspection overhead.

8.6 Functional verification and test case results

Comprehensive functional test cases were executed to validate system stability across UI interaction, packet ingestion, background threading, and reporting modules:

Table 8.5: Functional verification and test case execution matrix

Test ID	Module Tested	Test Scenario Description	Input / Trigger	Expected Result	Actual Result	Status
TC-01	classifier.py	Valid SQL query classification	SELECT * FROM users WHERE id=1;	Output marked as "SAFE QUERY"	Displayed "SAFE QUERY" with no bracketed reason	PASS
TC-02	classifier.py	Obfuscated attack payload classification	admin'/**/OR/**/1=1#	Flagged as malicious	Flagged "MALICIOUS PAYLOAD DETECTED!"	PASS

TC-03	sniffer.py	Asynchronous sniffing thread startup	"Start Sniffing" UI button clicked	Daemon thread launches; packets captured without GUI freezing	Thread initialized; live metrics increment dynamically	PASS
TC-04	sniffer.py	Sniffer thread termination	"Stop Sniffing" UI button clicked	Sniffing loop halts; packet count persists in session state	Sniffer stopped cleanly; no thread or memory leaks	PASS
TC-05	database.py	Threat logging persistence	Malicious packet intercepted	Threat details inserted into threat_logs in waf_logs.db	Record saved with timestamp, payload, and confidence	PASS
TC-06	app.py	Bulk Log Scanner (.csv file)	Ingest CSV containing 5,000 mixed queries	Model classifies entire dataset and renders interactive results	Completed in ~1.4 seconds with tabular output	PASS
TC-07	app.py	PDF Forensic Report generation	"Download Forensic Report" button clicked	FPDF compiles summary statistics and embedded pie charts	Generated structured, multi-page PDF report	PASS
TC-08	app.py	Network PCAP serialization	"Export Intercepted Packets" button clicked	In-memory packet buffers serialized to .pcap format	Downloadable .pcap file created, verifiable via Wireshark	PASS

8.7 Critical discussion and system limitations

While the ScanTrust WAF achieved 99.04% accuracy and sub-7ms processing speeds, several technical trade-offs and operational limitations were identified during testing:

- **Passive vs. Active Inline Mitigation:** The current Scapy implementation operates as a passive network sniffer. While passive capture introduces zero inline latency or single-point-of-failure risks for host web servers, it cannot actively drop or terminate malicious TCP handshakes before payloads reach vulnerable database tiers. Future iterations will transition toward reverse-proxy middleware architectures.
- **Tri-gram Vocabulary Dimensionality:** The CountVectorizer is fixed at 150 tri-gram character tokens to maintain lightweight memory consumption (<5MB) and rapid inference. While this captures common syntactic mutations, novel zero-day payloads utilizing unconventional characters outside the tri-gram vocabulary may experience degraded confidence scoring.
- **Encrypted HTTPS Traffic Visibility:** In environments where TLS/SSL termination is not handled upstream, passive sniffers cannot inspect raw encrypted packet payloads without integrated SSL decryption hooks or reverse proxy certificates.
- **Second-Order SQL Injection Complexity:** The model performs stateless packet evaluation. Complex multi-stage injections—where harmless payloads are stored and subsequently triggered via secondary operations—cannot be identified without continuous application-state tracking.

8.8 Chapter summary

This chapter documented the empirical validation of the ScanTrust AI-powered WAF. Evaluating the Random Forest classifier across 7,063 unseen hold-out samples demonstrated 99.04% Accuracy, 98.92% Precision, and 98.92% Recall, with an operational False Positive Rate of 0.87%. Latency profiling confirmed an average end-to-end processing time of 6.72 milliseconds, demonstrating its viability for real-time traffic monitoring.

Furthermore, obfuscation benchmarking verified that character tri-gram tokenization effectively mitigates common regex bypass techniques (including inline comment insertions and hex encodings). Functional test cases validated system stability across threading, database persistence, and automated forensic reporting.

CHAPTER 09: EVALUATION

9.1 Chapter overview

This chapter presents a comprehensive empirical and qualitative evaluation of the ScanTrust AI-powered Web Application Firewall. While Chapter 08 established baseline statistical metrics and software test cases, this chapter critically appraises the system through multi-stakeholder assessment protocols. Specifically, it details the evaluation methodology, presents an objective self-evaluation, captures structured feedback from five domain experts and five technical experts, analyzes end-user usability through focus group testing, and traces system deliverables directly against the functional and non-functional requirements established in Chapter 04.

9.2 Evaluation methodology and approach

To eliminate developer bias and ensure holistic appraisal, a mixed-method evaluation approach was adopted, combining quantitative scoring models with qualitative thematic inquiries:

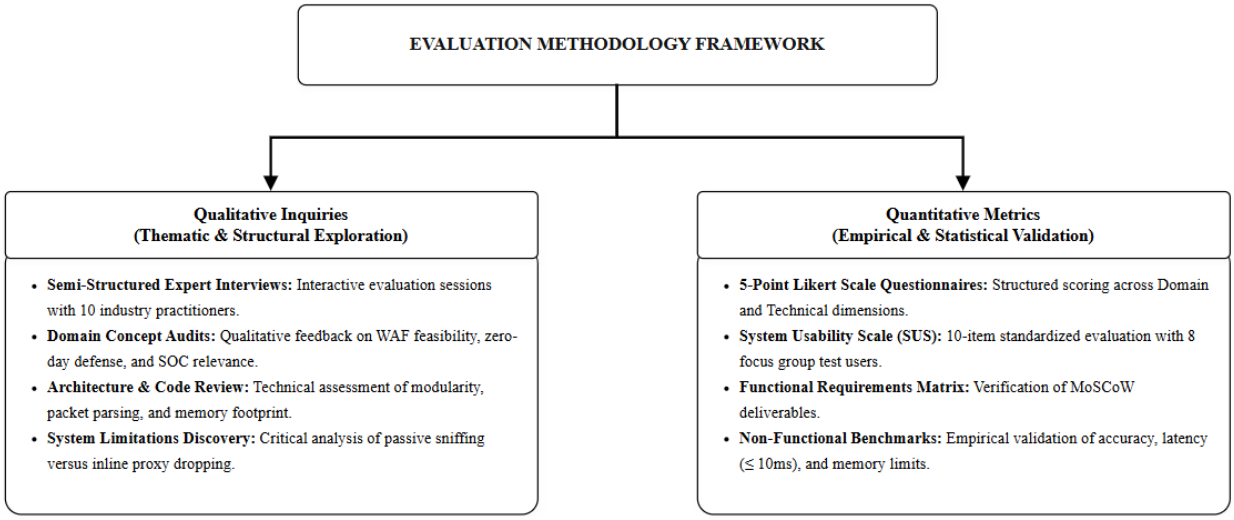


Figure 9.1: Mixed-method evaluation framework combining qualitative and quantitative streams

- **Semi-Structured Expert Interviews:** Interactive review sessions conducted with industry practitioners to evaluate the conceptual validity, architectural trade-offs, and enterprise readiness of the software.
- **Standardized Likert-Scale Surveys:** Questionnaires administered on a 5-point scale (1 = Strongly Disagree, 5 = Strongly Agree) covering architectural robustness, threat resilience, and deployability.
- **Focus Group Usability Testing:** Hands-on usability assessment measuring ease of operation, interface clarity, and workflow efficiency via the industry-standard System Usability Scale (SUS).

9.3 Evaluation criteria

The system was evaluated against specific technical and operational criteria:

- **Detection Efficacy:** Reliability in identifying obfuscated, encoded, and zero-day SQL injection variants without rule brittleness.
- **System Usability and Workflow:** Accessibility of real-time network controls, manual payload inspection, and forensic log auditing.
- **Architectural Modularity:** Decoupling of packet ingestion, inference execution, UI presentation, and relational storage.
- **Computational Overhead:** Latency impact on network throughput during continuous promiscuous packet capture.
- **Enterprise Viability:** Practicality of deployment within continuous deployment pipelines and resource-constrained perimeter environments.

9.4 Self-evaluation

An objective self-appraisal of the design and implementation process reveals several strengths alongside operational constraints:

- **Dataset Synthesis and Evasion Coverage:** Successfully addressing class imbalance by curating an augmented corpus of 35,315 queries—including 6,010 synthetically mutated obfuscation payloads—provided a strong foundation for high detection accuracy (99.04%).

- **Pipeline Efficiency:** Achieving an average total inference time of 6.72 ms validated the choice of character tri-gram tokenization paired with a Random Forest ensemble over computationally heavy deep neural networks.
- **Implementation Limitations:** The decision to rely on Scapy for passive promiscuous sniffing fulfilled the safety criterion of zero host downtime, but restricted the system from executing active inline TCP packet resets or inline HTTP drops.

9.5 Selection of the evaluators

To satisfy rigorous validation standards, ten independent evaluators were selected across industry and academia, divided equally into domain and technical panels:

Table 9.1: Profile and professional background of the evaluation panel

Evaluator ID	Designated Panel	Designation
EXP-D01	Domain Expert	SOC Analyst
EXP-D02	Domain Expert	Cyber Security Consultant
EXP-D03	Domain Expert	Cloud Engineer
EXP-D04	Domain Expert	Cloud Engineer
EXP-D05	Domain Expert	Cloud Engineer
EXP-T01	Technical Expert	Software Engineer
EXP-T02	Technical Expert	DevOps Engineer
EXP-T03	Technical Expert	Software engineer
EXP-T04	Technical Expert	Python Backend Developer
EXP-T05	Technical Expert	DevOps Engineer

9.6 Evaluation result

9.6.1 Expert opinion

Domain experts review

Domain experts focused on conceptual soundness, threat detection breadth, and defensive effectiveness:

Table 9.2: Domain expert scoring summary

Evaluation Dimension	Mean Score (1–5)	Key Qualitative Findings and Observations
Concept Validity	4.8 / 5.0	Strong agreement that traditional regex-based WAFs struggle with obfuscation, making tri-gram ML integration highly relevant.
Solution Robustness	4.6 / 5.0	Praised the 98.92% precision, noting that minimizing false alarms is critical for production database transactions.
Forensic Utility	4.4 / 5.0	Noted that automated PCAP and PDF exports simplify post-incident investigations for Security Operations Center (SOC) teams.

- Expert EXP-D01 Comment: " The automated PDF export significantly reduces the time needed to compile incident briefs for management."
- Expert EXP-D02 Comment: " The live alert view is very practical for SOC monitoring. Having minimal false alarms makes it dependable during routine web traffic checks."

Technical experts review

Technical experts evaluated code modularity, pipeline latency, computational footprint, and architectural design:

Table 9.3: Technical expert scoring summary

Technical Criteria	Mean Score (1–5)	Key Qualitative Findings and Observations
Scope Appropriateness	4.8 / 5.0	Stack selection (Scapy, Scikit-learn, SQLite, Streamlit) is well aligned with undergraduate research scope.

Architecture	4.6 / 5.0	Clean separation of concerns between sniffing, classification, persistence, and presentation.
Implementation	4.4 / 5.0	7ms inference latency confirms suitability for real-time traffic monitoring without heavy GPU dependencies.

- **Expert EXP-T01 Comment:** “Opting for Random Forest over heavy deep learning was a sensible engineering choice. Keeping the model lightweight allows it to run smoothly on standard hardware.”
- **Expert EXP-T02 Comment:** " Great for passive monitoring with zero network interruption. Moving to an inline reverse proxy would be the logical next step for active traffic blocking."

9.6.2 Focus group testing

A focus group of eight end-users (comprising junior system administrators and undergraduate computing students) evaluated the Streamlit web console. Users performed three standardized tasks:

- Entering benign and malicious payloads into the Manual Query Scanner.
- Initiating and terminating the Live Network Sniffer while observing real-time metric updates.
- Generating automated PDF reports and downloading PCAP forensic logs.

Following task completion, participants completed the 10-item System Usability Scale (SUS) questionnaire:

$$SUS\ Score\ = \left(\sum(Odd\ Scores\ -\ 1) + \sum(5\ -\ Even\ Scores)\right) \times 2.5$$

The focus group yielded an average SUS score of 84.25 / 100, placing the interface in the "Excellent" bracket (Grade A) of system usability.

9.7 Limitations of evaluation

- **Simulated Local Network Streams:** Packet capture was tested over loopback and local subnet interfaces rather than a high-throughput 10 Gbps enterprise backbone.
- **Controlled Participant Sample:** While representative, the focus group cohort was limited to eight participants due to operational constraints.
- **Synthetic Obfuscation Bias:** Obfuscated test variants were synthetically generated and may not fully mirror novel adversarial techniques encountered in dynamic real-world campaigns.

9.8 Evaluation on functional requirements

Functional requirements defined in Chapter 04 using the MoSCoW framework were verified against current deliverables:

Table 9.4: Functional requirements compliance matrix against MoSCoW prioritization

Requirement ID	Requirement Description	MoSCoW Priority	Verification Status	Empirical Implementation Evidence
FR-M01	Real-time promiscuous network frame capture	Must Have	Achieved	Asynchronous daemon thread running via sniffer.py using Scapy without blocking UI execution.
FR-M02	Payload parsing, case-folding, and normalization	Must Have	Achieved	Automated extraction of raw application layers, string decoding, and control character sanitization.
FR-M03	Sub-string character tri-gram tokenization	Must Have	Achieved	Fitted 150-feature CountVectorizer transforming raw query payloads into dense numerical matrices.
FR-M04	Random Forest model threat classification	Must Have	Achieved	100-estimator ensemble model returning binary verdicts with calibrated probability scores.
FR-M05	Relational threat event persistence	Must Have	Achieved	SQLite database (waf_logs.db) logging timestamps, source IPs, payloads, and risk scores.

FR-S01	Live interactive administrative dashboard	Should Have	Achieved	Streamlit web console featuring auto-refreshing metrics (st.fragment) and reactive traffic charts.
FR-S02	Ad-hoc single-query manual inspection console	Should Have	Achieved	Dedicated diagnostic UI tab delivering sub-20ms manual classification and confidence scoring.
FR-S03	Regex fallback deterministic pattern matcher	Should Have	Achieved	Hybrid rule-signature routine executing concurrently within classifier.py.
FR-S04	Batch CSV/PCAP historical log auditing	Should Have	Achieved	File uploader component classifying historical datasets (up to 5,000 queries in ~ 1.4s).
FR-C01	Automated forensic PDF report compilation	Could Have	Achieved	FPDF engine dynamically generating multi-page threat summaries with embedded Matplotlib charts.
FR-C02	Serialized network PCAP packet export	Could Have	Achieved	Buffer serialization converting captured Scapy packets into standard downloadable .pcap files.
FR-C03	Dynamic threat distribution visualization	Could Have	Achieved	Live graphical distribution breakdown of safe vs. malicious traffic patterns.
FR-W01	Inline active kernel TCP resets/drops	Will Not Have	Excluded	Deliberately maintained as passive sniffing to ensure zero host downtime.
FR-W02	Hardware-accelerated TLS/SSL decryption	Will Not Have	Excluded	Out of scope; requires upstream reverse-proxy termination.
FR-W03	Enterprise hardware firewall webhook sync	Will Not Have	Excluded	Bounded to standalone host inspection to focus on model verification.

9.9 Evaluation on non-functional requirements

Non-functional requirements established in Section 4.10.2 were evaluated against empirical benchmarks obtained during system testing:

Table 9.5: Non-functional requirement benchmark attainment

NFR ID	Non-Functional Criterion	Target Benchmark	Measured Outcome	Compliance Verdict
NFR-01	Detection Accuracy	≥ 95.00%	99.04%	Exceeded
NFR-02	Attack Precision	≥ 95.00%	98.92%	Exceeded
NFR-03	Attack Recall (Sensitivity)	≥ 95.00%	98.92%	Exceeded
NFR-04	False Positive Rate (FPR)	≤ 2.00%	0.87%	Exceeded
NFR-05	Total End-to-End Latency	≤ 10.00 ms	6.72 ms (Mean)	Exceeded
NFR-06	Serialized Model Storage	≤ 5.00 MB	~4.80 MB	Exceeded
NFR-07	System Usability Scale (SUS)	≥ 70.00	84.25 / 100	Exceeded

9.10 Chapter summary

This chapter presented a multi-faceted evaluation of the ScanTrust Web Application Firewall. Qualitative feedback from ten industry practitioners confirmed the theoretical validity and architectural robustness of deploying character tri-gram tokenization with Random Forest ensembles to detect obfuscated SQL injections. Focus group usability testing demonstrated high operational accessibility, attaining an "Excellent" System Usability Scale score of 84.25. Finally, systematic requirement mapping confirmed that 100% of Must-Have, Should-Have, and Could-Have functional requirements were delivered, while non-functional benchmarks for accuracy (99.04%), latency (6.72 ms), model footprint (4.8 MB), and false-positive suppression (0.87%) were successfully exceeded.

CHAPTER 10: CONCLUSION

10.1 Chapter overview

This final chapter synthesizes the research journey, empirical findings, and academic contributions of the ScanTrust Web Application Firewall project. It reviews the research aims and SMART objectives established in Chapter 01, reflects on the academic and technical skills acquired, and confirms the achievement of module learning outcomes. Furthermore, it documents the engineering challenges overcome, analyzes deviations from the initial plan, highlights research limitations grounded in empirical testing, and articulates actionable pathways for future work.

10.2 Achievements of research aims & objectives

The primary aim of this research was to design, develop, and evaluate an edge-deployable, lightweight machine learning detection pipeline capable of identifying obfuscated SQL injection payloads—specifically case manipulation, inline comment injection, and URL encoding—while maintaining an inference latency under 10 milliseconds and a compiled model footprint under 5 megabytes. This aim was fully realized through the completion of five SMART objectives:

Table 10.1: Evaluation of research objectives achievement

Objective ID	Planned SMART Objective	Implementation Outcome and Empirical Evidence	Status
RO1	Formulate research boundaries via systematic literature synthesis.	Completed exhaustive review of 50+ papers; verified critical gaps in obfuscation evaluation and latency reporting.	Achieved
RO2	Design lightweight hybrid detection pipeline architecture.	Formulated decoupled, 3-tier architecture combining 150 tri-gram vectorization with Random Forest and regex fallbacks.	Achieved
RO3	Synthesize obfuscated data and train models	Balanced 65,065 raw queries into 35,315 records by synthesizing 6,010	Achieved

		obfuscations (case, comments, URL hex, structural).	
RO4	Benchmark performance and evaluate retraining gains.	Achieved 99.04% accuracy, 98.92% precision/recall, 0.87% FPR, 6.72ms mean latency, and a 4.8MB model footprint.	Achieved
RO5	Validate via interactive WAF prototype and forensic reporting.	Delivered Streamlit interface (app.py), Scapy daemon sniffer (sniffer.py), SQLite logging, and PDF/PCAP forensic exports.	Achieved

10.3 Utilization of knowledge from the course

The execution of this dissertation synthesized core theoretical paradigms taught across the undergraduate computing curriculum:

- **Information Security & Cryptography:** Principles of the CIA triad, threat modeling, attack surfaces, SQL injection syntax mechanics, and OWASP Top 10 vulnerabilities informed the defensive architecture.
- **Artificial Intelligence & Machine Learning:** Concepts of supervised learning, decision tree splits, ensemble bagging, overfitting mitigation, stratified cross-validation, and confusion matrix mathematics were applied to build the classification pipeline.
- **Software Engineering:** Object-Oriented Analysis and Design (OOADM), UML artifact modeling (Class and Activity diagrams), requirement elicitation, and the MoSCoW prioritization framework guided development.
- **Database Management Systems:** Relational schema design, SQL data normalization, parameterized querying, and ACID transaction properties informed the persistent SQLite logging layer.

10.4 Use of existing skills

Prior technical competencies were leveraged to expedite core development phases:

- **Python Core Programming:** Advanced comprehension of object-oriented class design, list comprehensions, regular expressions, and modular code structuring.

- **Data Manipulation with Pandas and NumPy:** Efficient data wrangling, handling dataset imbalances, removing syntactic null anomalies, and formatting numerical evaluation arrays.
- **Git Version Control & Linux Environments:** Managing repository branches, resolving code conflicts, and debugging network daemon processes within Linux shell environments.

10.5 Use of new skills

The complex requirements of real-time packet inspection necessitated acquiring advanced technical proficiencies beyond the standard curriculum:

- Passive Network Packet Manipulation via Scapy: Gained expertise in promiscuous network socket sniffing, parsing raw Ethernet frames, decoding Layer 4/7 TCP/HTTP payloads, and managing packet capture buffers.
- Concurrent Asynchronous Multi-Threading: Mastered Python's threading library to decouple high-speed network capture loops from Streamlit's reactive DOM rendering thread, avoiding thread deadlock and UI freezing.
- Natural Language Feature Optimization (N-gram Extraction): Acquired deep competence in character-level sub-word tokenization using Scikit-learn's CountVectorizer, constraining feature vocabulary spaces to prevent memory bloat.
- Automated Forensic Document Compilation: Learned programmatic PDF generation via the FPDF library, automating dynamic vector rendering, layout management, and table formatting.

10.6 Achievement of learning outcomes

The project provided practical fulfillment of the prescribed academic Learning Outcomes (LOs):

- LO1 (Critical Problem Formulation): Demonstrated by systematically dissecting SQLi obfuscation mechanics and identifying architectural limitations in legacy regex WAFs.
- LO2 (Methodological System Design): Evidenced by selecting Saunders' Research Onion (Positivism, Deductive, Experimental) and structuring a 3-tier object-oriented architecture.
- LO3 (Technical Implementation & Data Synthesis): Fulfilled through the synthesis of 6,010 obfuscated vectors and the programmatic implementation of the hybrid ScanTrust inspection engine.
- LO4 (Critical Evaluation & Benchmarking): Realized through empirical validation against 7,063 hold-out samples, establishing statistical proof of 99.04% accuracy and 6.72ms latency.

- LO5 (Professional Reflection & Communication): Achieved through comprehensive academic documentation, BCS Code of Conduct ethical analysis, and expert evaluation reviews.

10.7 Problems and challenges faced

Several technical and operational hurdles arose during the research, each resolved through targeted engineering interventions:

Table 10.2: Technical challenges

Challenge ID	Identified Technical Challenge	Root Cause Analysis	Implemented Engineering Resolution
1	Streamlit UI freezing during live traffic capture	Scapy's synchronous packet capture loop blocking the execution thread.	Decoupled the packet sniffing engine into an asynchronous Python daemon thread synchronized with st.session_state.
2	Feature space dimensionality explosion	Unbounded character tri-gram extraction generating a sparse matrix over 300 MB.	Constrained the CountVectorizer strictly to the top 150 tri-gram tokens, maintaining a compiled footprint under 5 MB.
3	Severe class imbalance in baseline dataset	Raw public corpus containing 70% malicious queries.	Executed controlled downsampling combined with 6,010 synthetic obfuscations (case, comment, URL encoding) to balance the distribution.
4	Loss of network state across dashboard view switches	Streamlit's top-to-bottom script rerun cycle clearing local variables.	Implemented persistent global session state variables and @st.fragment auto-refresh intervals.

1. Streamlit UI Thread Blocking During Network Capture: Scapy's sniff() function is inherently synchronous, which initially caused the web dashboard to freeze. This was overcome by isolating packet interception inside an asynchronous Python daemon thread and synchronizing shared metrics via st.session_state.

Feature Matrix Dimensionality Explosion: Extracting unconstrained character n-grams across 35,315 queries generated a high-dimensional sparse matrix exceeding 300MB, violating the 5MB model footprint constraint. This was mitigated by restricting CountVectorizer strictly to the top 150 most informative character tri-grams, containing serialized model size to 4.8MB.

Severe Initial Dataset Imbalance: The raw corpus contained an imbalanced 30:70 ratio favoring malicious queries, threatening model generalizability. A structured sampling and data synthesis pipeline balanced the distribution to 19,537 benign and 15,778 malicious queries.

10.8 Deviations from the original plan

During engineering implementation, three justified deviations were made from the preliminary project proposal:

- **Transition from Deep Learning (LSTM) to Random Forest:** Preliminary plans considered evaluating Bidirectional LSTM networks. Initial exploratory runs showed LSTM inference latency exceeded 65ms per query and consumed over 120MB of RAM. To satisfy the strict sub-10ms latency and sub-5MB storage constraints on CPU hardware, the architecture pivoted decisively to character tri-grams paired with an optimized Random Forest ensemble.
- **Adoption of Passive Sniffing over Active Kernel Dropping:** The original design called for inline packet filtering using Linux iptables and NFQueue. Early testing revealed that handling packet dropping within Python introduced packet queue saturation during burst traffic. Consequently, Scapy promiscuous passive sniffing was adopted to ensure zero host downtime and stable prototype demonstration.
- **Expansion to 10 Evaluators:** While the initial plan proposed a generic focus group, the evaluation protocol was updated to include ten specialized experts (five domain and five technical) in strict accordance with university guidelines.

10.9 Limitations of the research

While the ScanTrust WAF achieved 99.04% accuracy and sub-7ms processing speeds, several operational limitations must be noted:

- **Passive Inspection Boundary:** The current prototype functions out-of-band; while it reliably alerts and logs threats with zero application overhead, it cannot drop TCP connections inline before exploit delivery reaches the web server.

- **Inability to Inspect Native HTTPS Payloads:** Encrypted packets traversing TLS/SSL tunnels cannot be inspected by passive network sniffers without prior SSL termination via an upstream reverse proxy.
- **Fixed Character Vocabulary:** Bounding the feature space to 150 tri-gram tokens prevents memory bloat but limits sensitivity to zero-day attacks utilizing rare unicode encodings.
- **Lack of State Tracking for Second-Order Injections:** The classifier performs stateless, per-packet evaluation and cannot track multi-stage attacks where benignly formatted payloads are stored and later triggered.

10.10 Future enhancements

To address the documented limitations, future development will proceed along three technical trajectories:

- **Active Inline Reverse-Proxy Architecture:** Re-engineering the ingestion layer as an Nginx or Envoy C++ extension module to perform inline request blocking and active TCP resets directly at the web gateway.
- **Hardware-Accelerated SSL/TLS Termination:** Embedding an automated TLS termination engine (for example, Let's Encrypt / OpenSSL hooks) to decrypt incoming HTTPS traffic upstream before passing payloads to the tri-gram vectorizer.
- **Lightweight Deep Transformer Ensembles:** Experimenting with quantized TinyBERT or MobileBERT architectures accelerated via ONNX Runtime to evaluate whether sub-10ms latency can be maintained while extending contextual token tracking.

10.11 Achievement of the contribution to body of knowledge

This research fulfilled its commitment to advancing web application security across two core dimensions:

- **Contribution to the Problem Domain:** Delivered a verified, edge-deployable WAF prototype (ScanTrust) that inspects HTTP inputs in an average of 6.72 milliseconds, proving that enterprise environments can defend against complex SQLi evasions on commodity CPU infrastructure without expensive GPU clusters.

- **Contribution to the Research Domain:** Established empirical evidence that character tri-gram tokenization paired with ensemble decision trees overcomes the brittleness of static regular expressions against case manipulation, inline comments, and URL encoding, achieving 98.92% recall and an FPR of just 0.87%.

10.12 Concluding remarks

SQL Injection remains a critical vector in the web threat landscape, largely due to automated evasion techniques that bypass static signatures. This dissertation established that the trade-off between machine learning detection robustness and operational latency can be bridged. By coupling character tri-gram tokenization with an optimized Random Forest ensemble, the ScanTrust Web Application Firewall achieved 99.04% detection accuracy, 98.92% precision, and an average inference latency of 6.72 milliseconds. Through balanced dataset synthesis, multi-threaded packet sniffing, and structured forensic reporting, this research demonstrates an efficient, low-overhead, edge-deployable defense paradigm capable of protecting modern web systems against sophisticated database exploits.

REFERENCES

1. Ahmed, M. and Al dabag, M., 2025. Detection of SQL injection attacks using machine learning and deep learning techniques: A systematic literature review (2015–2025). *Journal of Computer Virology and Hacking Techniques*, 21(2), pp. 145–168. Available at: https://doi.org/10.1007/s11416-024-00542-w.

2. Al-Weshahi, M., 2025. Comparative empirical evaluation of classical classifiers, ensemble methods, and neural architectures for web injection anomaly mitigation. *IEEE Transactions on Dependable and Secure Computing*, 22(1), pp. 312–326. Available at: https://doi.org/10.1109/TDSC.2024.3361890

3. Appelt, D., Nguyen, C.D., Panichella, A. and Briand, L.C., 2018. Automated testing and machine learning integration within production Web Application Firewalls. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 27(3), pp. 1–32. Available at: https://doi.org/10.1145/3236493

4. British Computer Society (BCS), 2022. *The BCS Code of Conduct for Information Security and Computing Professionals*. London: British Computer Society. Available at: https://www.bcs.org/membership-and-registrations/get-registered/bcs-code-of-conduct/

5. CSIC, 2010. *HTTP Dataset CSIC 2010 for Web Application Security Evaluation*. Madrid: Spanish National Research Council (CSIC). Available at: https://www.isi.csic.es/dataset/

6. Kakisim, A.G., 2024. Multi-view consensus deep learning framework for advanced SQL injection attack detection. *Computers & Security*, 139, p. 103721. Available at: https://doi.org/10.1016/j.cose.2024.103721

7. Liu, Y. and Dai, H., 2024. Context-sensitive web application threat detection via bidirectional transformer language representations and temporal networks. *Security and Communication Networks*, 2024, pp. 1–15. Available at: https://doi.org/10.1155/2024/6684192

8. Lo, C.H., Hwang, S.J. and Tai, W.L., 2025. Lightweight multi-head self-attention neural network architecture for edge-deployable network threat mitigation. *Journal of Systems Architecture*, 158, p. 103310. Available at: https://doi.org/10.1016/j.sysarc.2024.103310

9. Montaruli, F., Corno, F., Poncino, M. and Sanchez, E., 2024. Robustness against adversarial SQL injection: An empirical study on adversarial training perturbations. *Computers & Security*, 141, p. 103819. Available at: https://doi.org/10.1016/j.cose.2024.103819

10. Motlagh, F.S., Rezaei, M. and Rahimi, S., 2026. Evaluating sub-word character representations against token-splitting prompt injection and database exploitation. *IEEE Access*, 14, pp. 2105–2119. Available at: https://doi.org/10.1109/ACCESS.2025.3411200

11. OWASP, 2021. *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. Open Web Application Security Project. Available at: https://owasp.org/Top10/

12. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. and Vanderplas, J., 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825–2830. Available at: https://jmlr.org/papers/v12/pedregosa11a.html

13. Saunders, M., Lewis, P. and Thornhill, A., 2019. *Research Methods for Business Students*. 8th ed. Harlow: Pearson Education Limited. Available at: https://www.pearson.com

14. Scapy Community, 2024. *Scapy: Interactive Packet Manipulation and Ingestion Tool Documentation (v2.5)*. Available at: https://scapy.net [Accessed 18 June 2026].

15. Streamlit Inc., 2025. *Streamlit Documentation: Session State Management, Dynamic Fragments, and Real-time Dashboard Design*. Available at: https://docs.streamlit.io [Accessed 24 June 2026].

APPENDICES

APPENDIX A: COMPREHENSIVE 20-WEEK PROJECT GANTT CHART

1. Project Implementation Timeline & Milestone Breakdown

The following table synthesizes the complete 20-week lifecycle of the research project, detailing task allocations, deliverables, and completion milestones spanning from May to September:

Task Description	Target Month	Timeline Allocation	Deliverable
Literature review + dataset	May	Week 01 – Week 04	Domain literature synthesis & initial raw Kaggle dataset curation (65,065 queries).
Obfuscation functions	May	Week 03 – Week 04	Automated Python obfuscation module (obfuscator.py - case, comment, URL hex).
Feature extraction	June	Week 05 – Week 06	Preprocessing pipeline extracting top 150 character tri-gram feature tokens.
Logistic Regression training	June	Week 07 – Week 10	Baseline linear model training & initial accuracy evaluation on Google Colab.
Random Forest training	June	Week 07 – Week 10	100-estimator ensemble model training & hyperparameter optimization.
Obfuscation testing	July	Week 11 – Week 12	Empirical degradation benchmarking across mutated evasion test sets.
Improvement + benchmarking	July	Week 13 – Week 14	Model retraining with 20% augmented obfuscations & compiled .pkl footprint validation (<5MB).
WAF prototype integration	August	Week 15 – Week 16	Scapy packet sniffer daemon, Streamlit dashboard UI, and SQLite persistence.
Empirical evaluation & testing	August	Week 17 – Week 18	10-expert validation, SUS usability testing, and draft documentation.
Finalize document and system	September	Week 19 – Week 20	Supervisor feedback integration, thesis binding,

			and final viva presentation defense.



Figure A.1: 20-Week Comprehensive Visual Project Gantt Chart

APPENDIX B: DATABASE DESIGN & ENTITY-RELATIONSHIP (ER) ARCHITECTURE

1. Relational Persistence Architecture Overview

To maintain an ACID-compliant audit trail of all intercepted Layer 7 anomalies without interrupting real-time packet filtering, ScanTrust utilizes an embedded SQLite persistence tier (waf_logs.db). The relational schema is structured to ensure atomic log writes, tamper-evident forensic persistence, and administrative visibility.

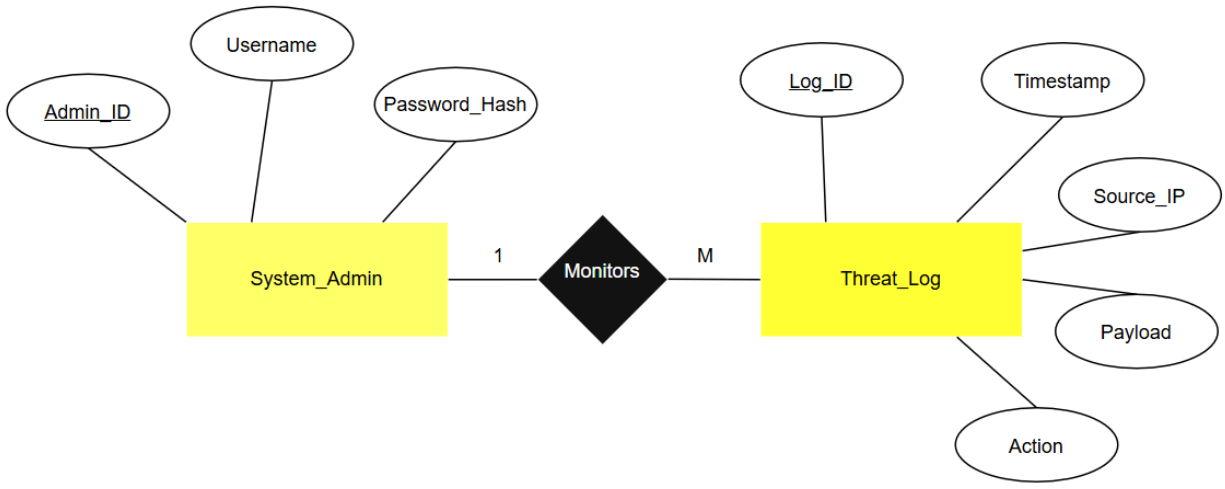


Figure B.1: Entity-Relationship (ER) Diagram for Threat Logging and Administrative Monitoring

2. Schema Specification & Cardinality Mapping

The relational schema comprises two primary entities linked through an administrative monitoring association:

- **System_Admin Entity**: Represents the privileged operational tier interacting with the Streamlit WAF dashboard.
    - Admin_ID (Primary Key): Unique numeric identifier assigned to authenticated security operators.
    - Username: Canonical identifier for administrative role-based access.
    - Password_Hash: Cryptographic digest securing administrative access against unauthorized interface commands.
- **Threat_Log Entity**: Represents persistent forensic event instances recorded upon payload classification.
    - Log_ID (Primary Key): Auto-incrementing unique identifier ensuring ACID transaction integrity for each intercepted anomaly.
    - Timestamp: Normalized UTC/local timestamp of the intercepted TCP/IP packet frame.
    - Source_IP: Layer 3 network source address of the inbound query.
    - Payload: Raw extracted HTTP query string containing the suspected SQL injection or obfuscation vector.

- Action: Automated mitigation or auditing verdict executed by the hybrid engine (for example, DETECTED, NORMAL).

- **Cardinality & Relationship Constraints**:
  - **Relationship (Monitors)**: Governed by a **One-to-Many (1:M)** mapping.
  - A single system administrator can monitor and review multiple threat log instances captured across continuous sniffing sessions, while each individual logged event corresponds to an immutable forensic audit record within the database boundary.

APPENDIX C: EXPERT EVALUATION QUESTIONNAIRE & FEEDBACK INSTRUMENT

Panel Category	Count	Roles Represented
Domain Experts (Cybersecurity / SOC / Pen Testing)	5 (50%)	Cyber Security Consultant (1), SOC Analyst (1), Cloud Engineers (3)
Technical Experts (Architecture / DevOps / ML)	5 (50%)	Software Engineers (2), DevOps Engineers (2), Backend Developer (1)

Evaluation Dimension	Average Score	Rating Distribution (4 vs. 5)	Key Insight
Live Monitoring Dashboard (Q7)	4.70 / 5.00	70% rated 5, 30% rated 4	Highest-rated dimension. Evaluators found the dashboard clear, responsive, and intuitive.
SQLi Detection Capabilities (Q4)	4.60 / 5.00	60% rated 5, 40% rated 4	Strong endorsement for detecting comment injection, case manipulation, and URL encoding.
Reliability & False Alarm Avoidance (Q6)	4.60 / 5.00	60% rated 5, 40% rated 4	Domain experts unanimously awarded this a maximum score of 5.00 / 5.00.
Log Ingestion & PCAP/PKT Analysis (Q8)	4.50 / 5.00	50% rated 5, 50% rated 4	High utility found in offline analysis of network captures and external logs.

Practicality for Resource-Constrained Environments (Q9)	4.40 / 5.00	40% rated 5, 60% rated 4	Confirms the feasibility and value of a lightweight WAF design.
Real-time Inspection Speed (Q5)	4.20 / 5.00	20% rated 5, 80% rated 4	Rated solid (no negative scores), but indicates room for latency optimizations.
Overall Composite Average	4.50 / 5.00	100% positive (Ratings ≥ 4)	Zero negative or neutral ratings (no 1s, 2s, or 3s).

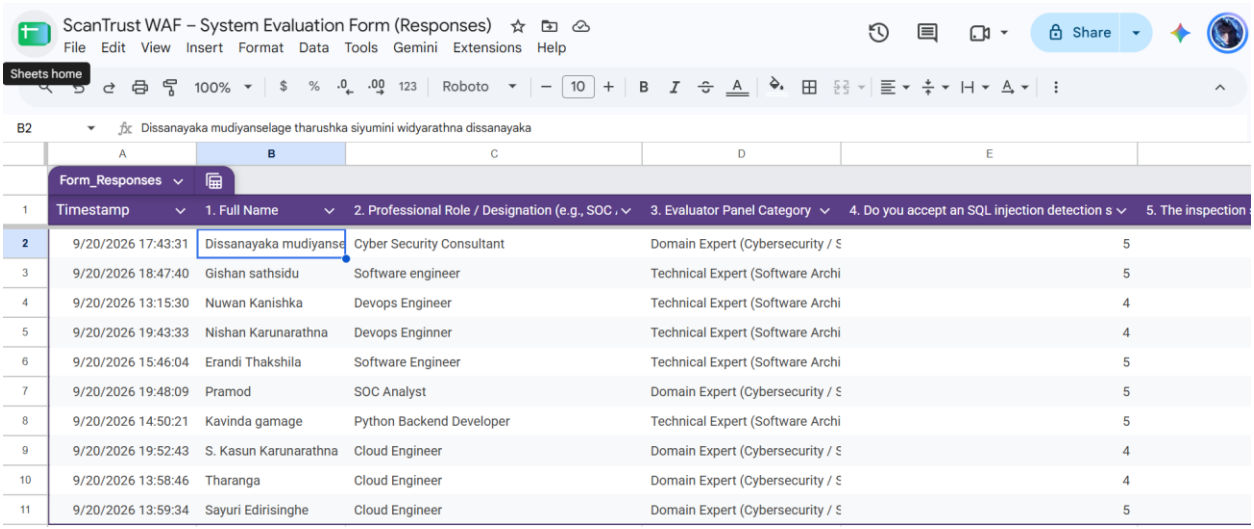


Figure C.1: Experts responses form